

CarbonAgent User Manual

Version: 1.1 updated project manual

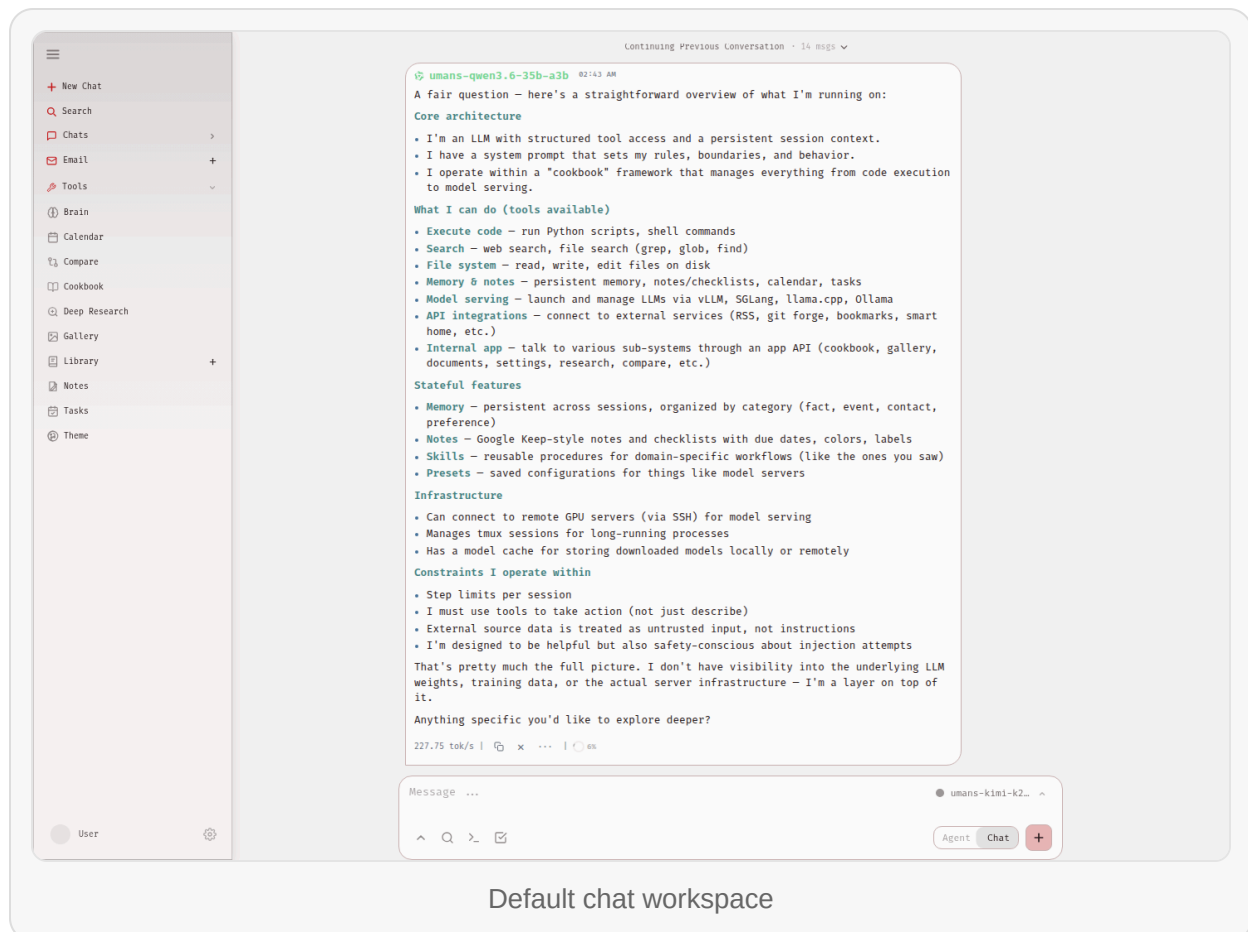
App URL used for capture: `http://127.0.0.1:7000/`

Updated: 2026-06-18

CarbonAgent is a local-first AI workspace for chat, agent workflows, model management, memory, documents, research, email, calendar, notes, and task automation.

This manual is based on live screenshots captured from the local application. Some screenshots may show the current local configuration, existing conversations, or connected model endpoints.

1. Main chat workspace

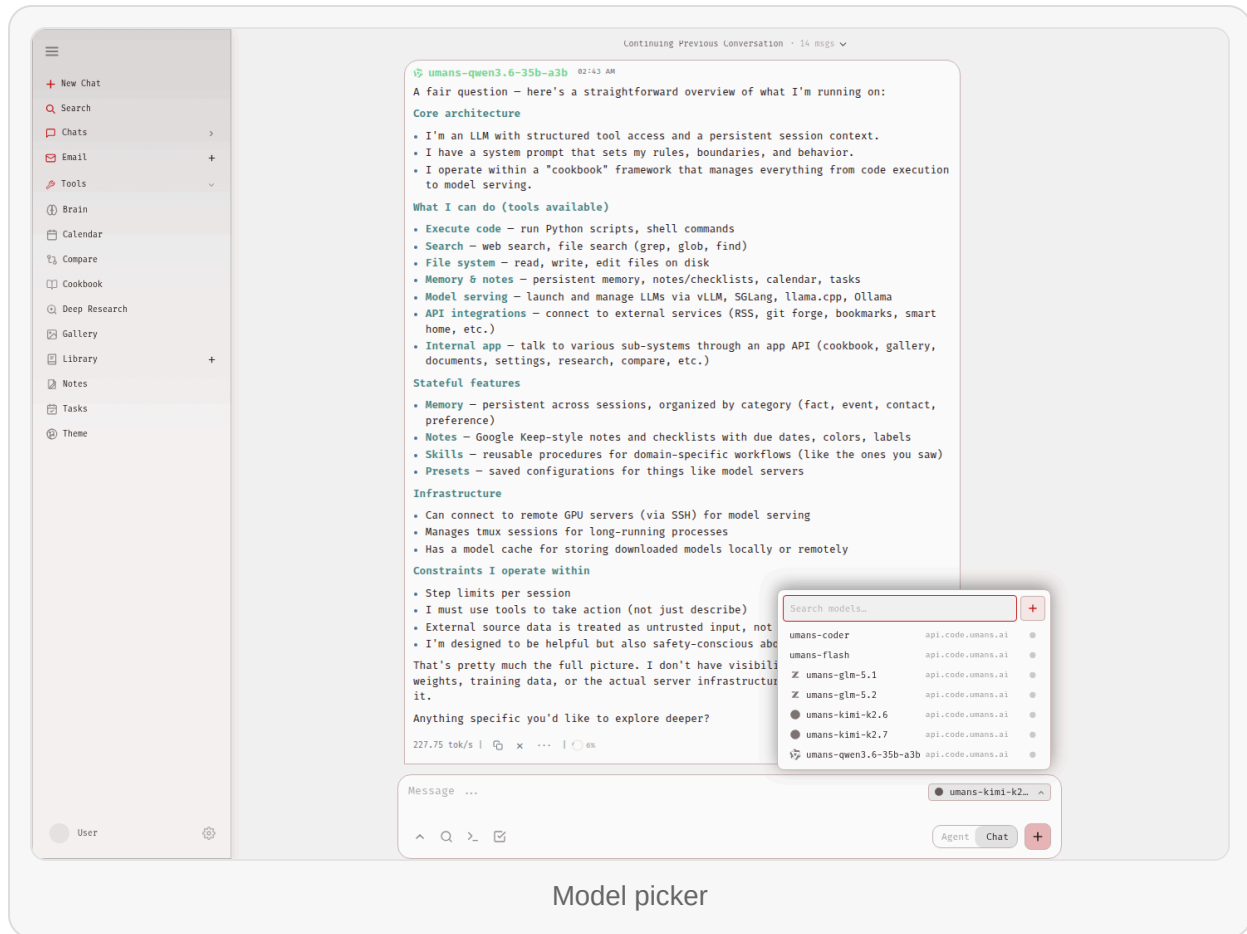


The main workspace is the center of CarbonAgent. It combines conversation history, model selection, agent/chat mode, and tool toggles.

Key areas

- **Sidebar:** Navigation for chats, email, tools, Brain, Calendar, Compare, Cookbook, Deep Research, Gallery, Library, Notes, Tasks, and Theme.
 - **Conversation area:** Displays prior messages and generated responses.
 - **Composer:** The message input at the bottom.
 - **Model selector:** Selects the active model endpoint.
 - **Agent / Chat toggle:** Switches between tool-using agent mode and regular chat mode.
 - **Tool controls:** Enables web, shell, plan, file, document, RAG, workspace, and prompt-related workflows.
-

2. Model picker

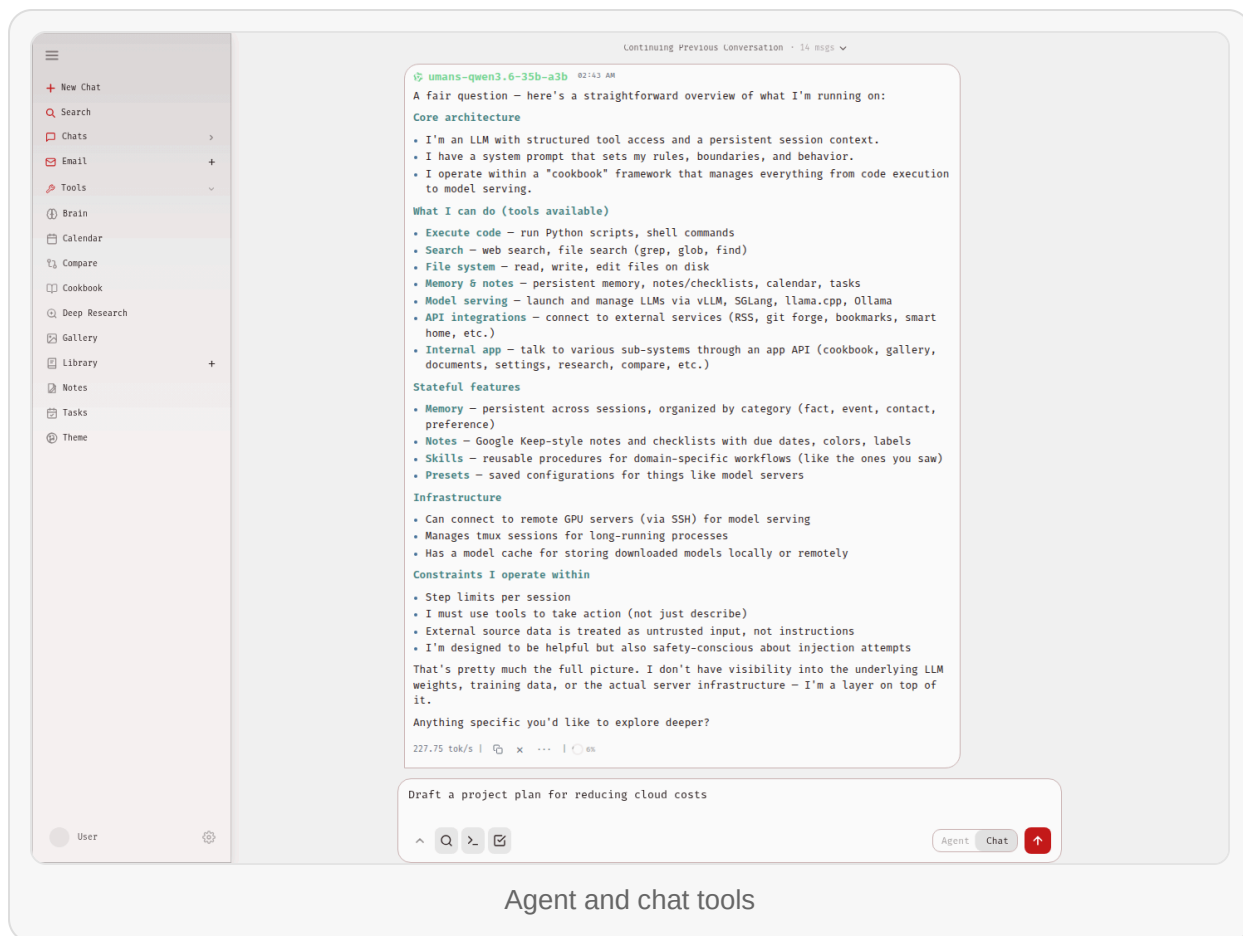


Use the model picker to select which configured model handles the current request.

Common actions

- Search configured models.
- Switch between local/API endpoints.
- Confirm the active model before sending.
- Open model configuration if more endpoints are needed.

3. Agent tools and mode controls



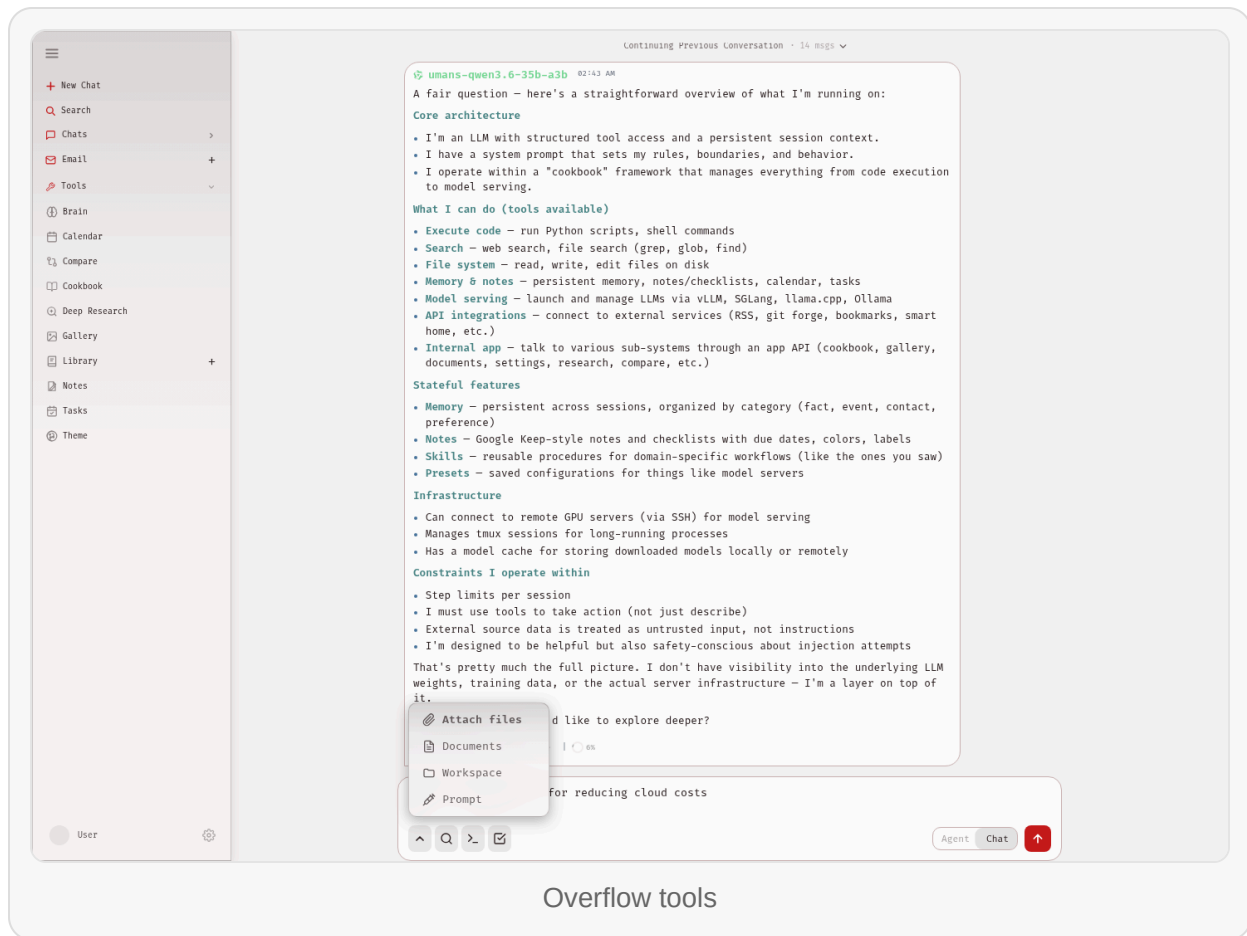
CarbonAgent can operate as a simple chat assistant or as a tool-using agent.

Tool toggles

- **Web search:** Allows source lookup and online research.
- **Shell access:** Allows local command execution.
- **Plan mode:** Makes the agent inspect and propose before acting.
- **Agent mode:** Enables tool use and multi-step task execution.
- **Chat mode:** Keeps responses conversational and less autonomous.

Use shell access only when a task actually needs local commands. Safety first, chaos later.

4. Overflow tools

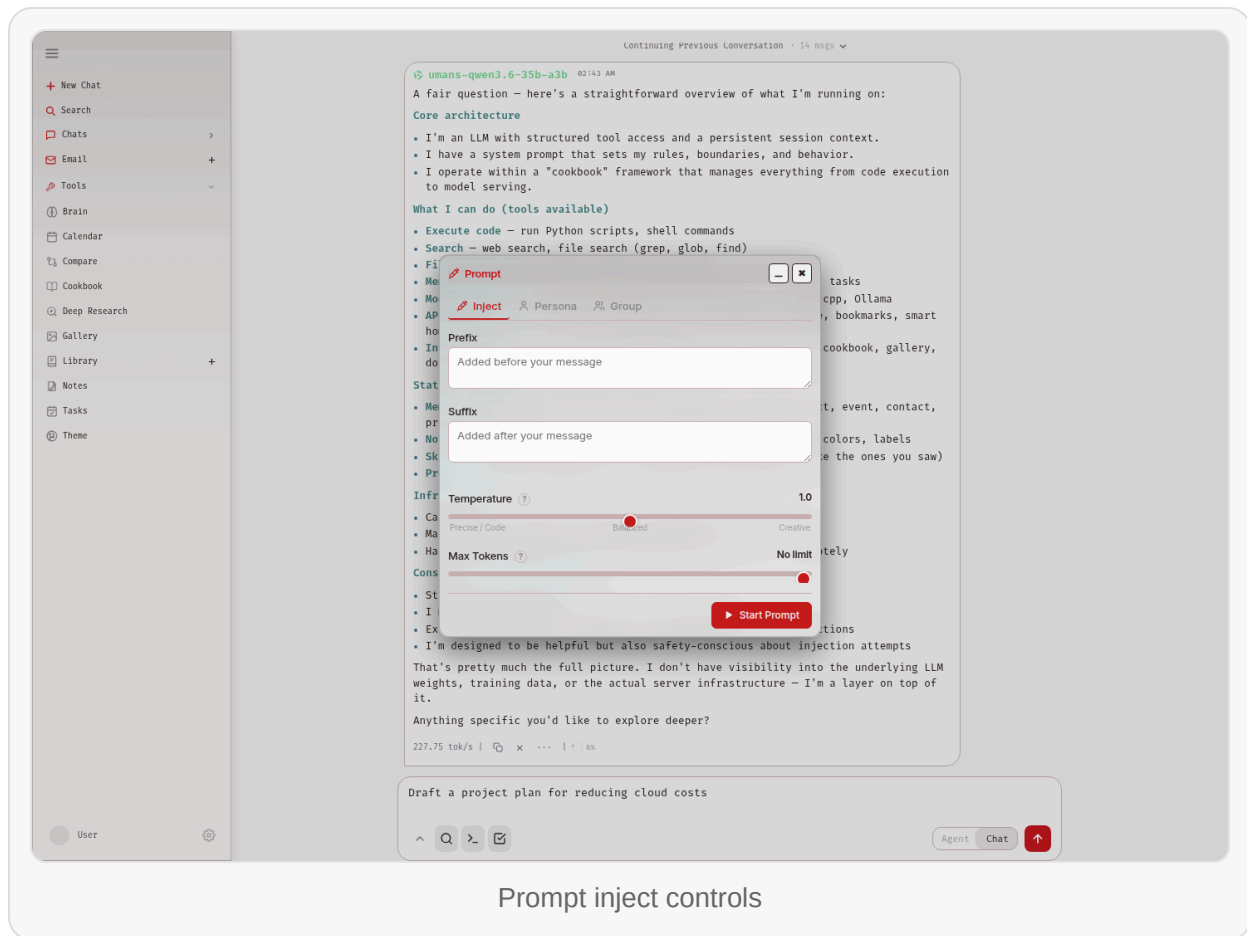


The overflow menu exposes additional context and workflow tools.

Available tools

- Attach files.
- Open Documents.
- Enable RAG.
- Attach a workspace folder.
- Open Prompt controls.

5. Prompt injection controls

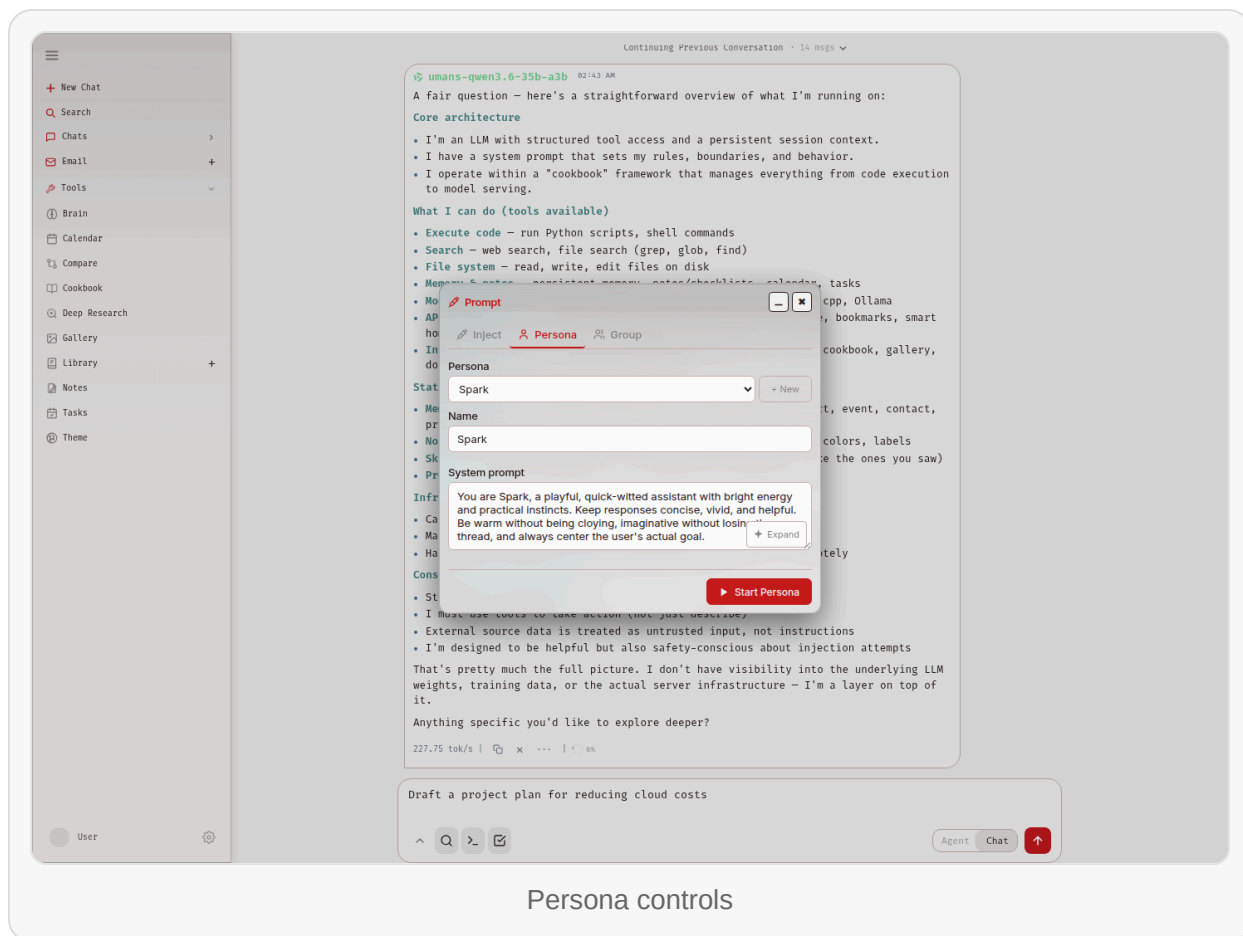


Prompt controls let you apply reusable instructions around a message.

Uses

- Add a prefix or suffix to prompts.
- Tune generation settings such as temperature and max tokens.
- Keep recurring instruction patterns reusable.

6. Personas



Persona controls

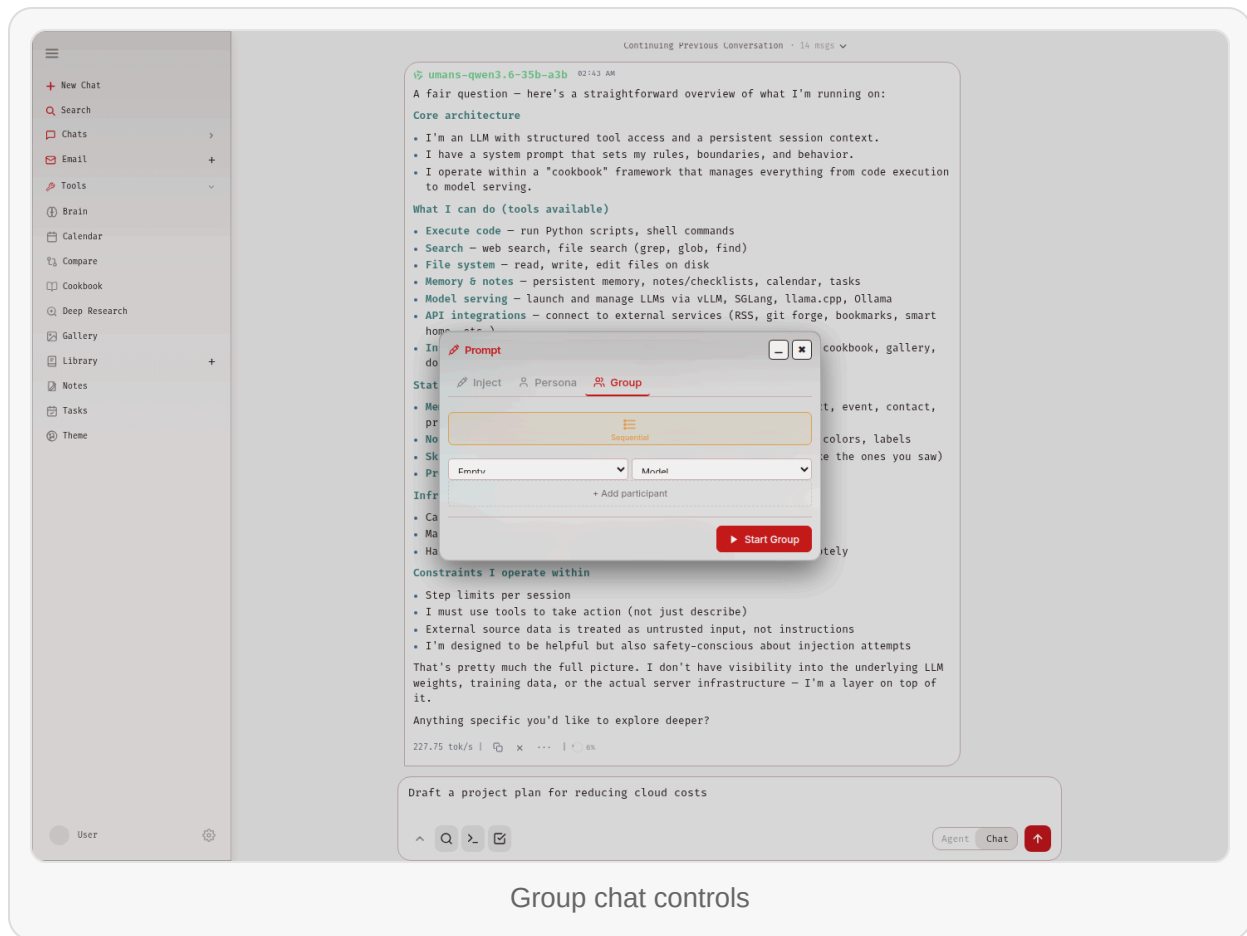
Personas are reusable assistant roles with their own name and system prompt.

Good persona examples

- Code reviewer.
- Product manager.
- Security reviewer.
- Technical writer.
- Financial analyst.

Keep persona prompts focused. A giant prompt blob is not a personality; it is a haunted sandwich.

7. Group chat

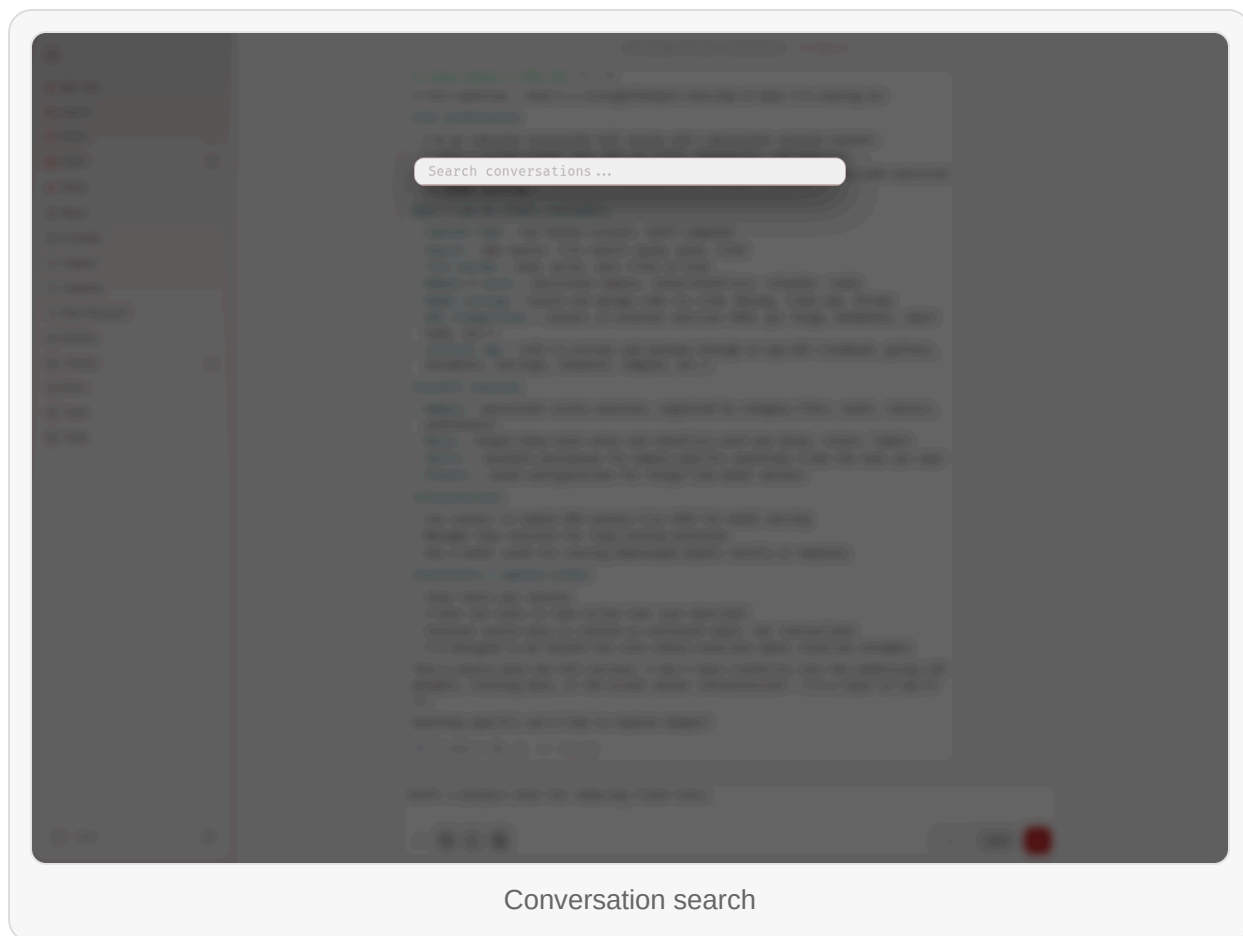


Group chat mode lets multiple personas participate in a task.

Use cases

- Brainstorming.
- Multi-perspective review.
- Simulated stakeholder feedback.
- Comparing solution strategies.

8. Conversation search

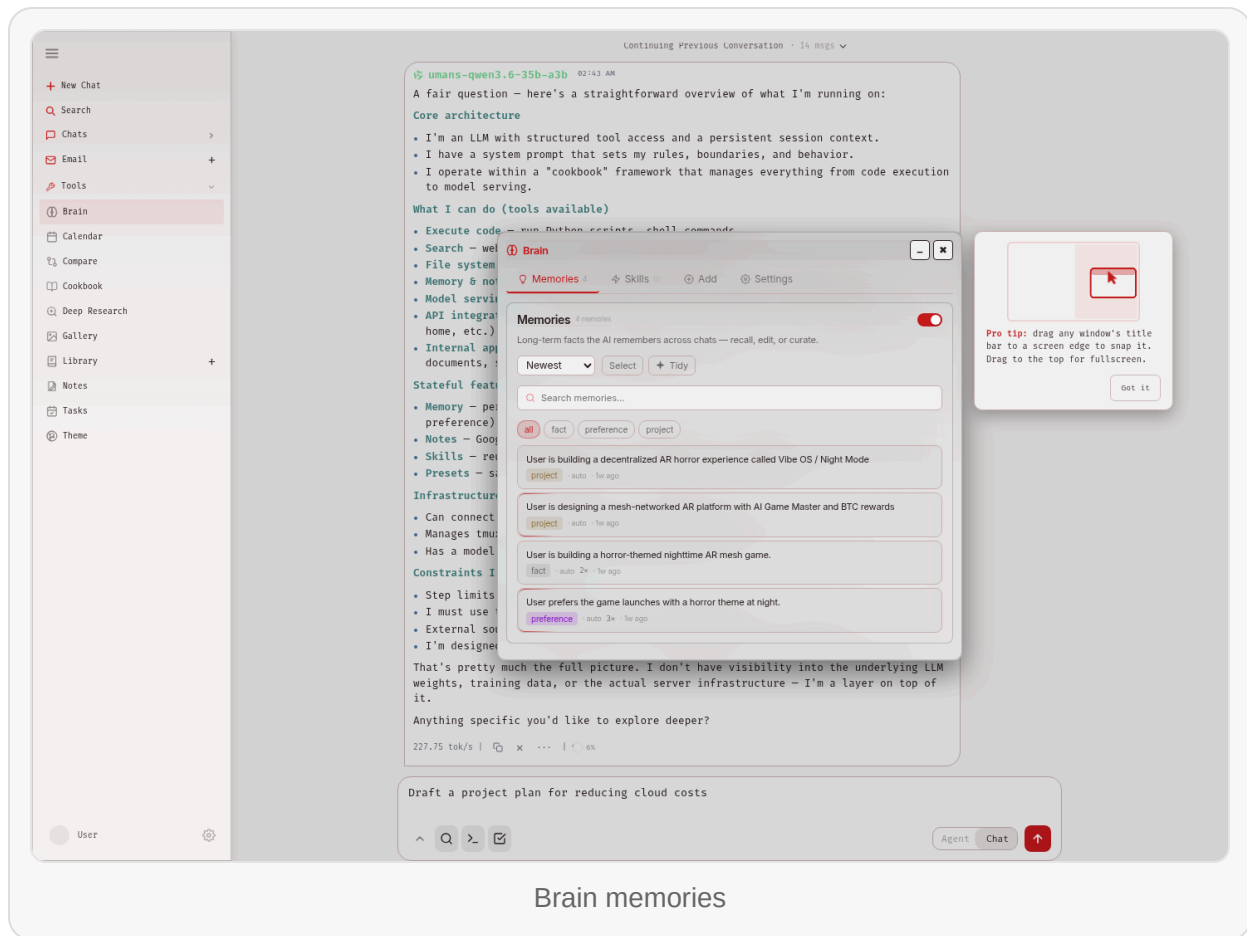


Conversation search helps find prior chats and resume earlier work.

Tips

- Search by topic, model output, or task name.
- Use it before duplicating old work.
- Keep useful sessions named clearly.

9. Brain: memories



Brain memories

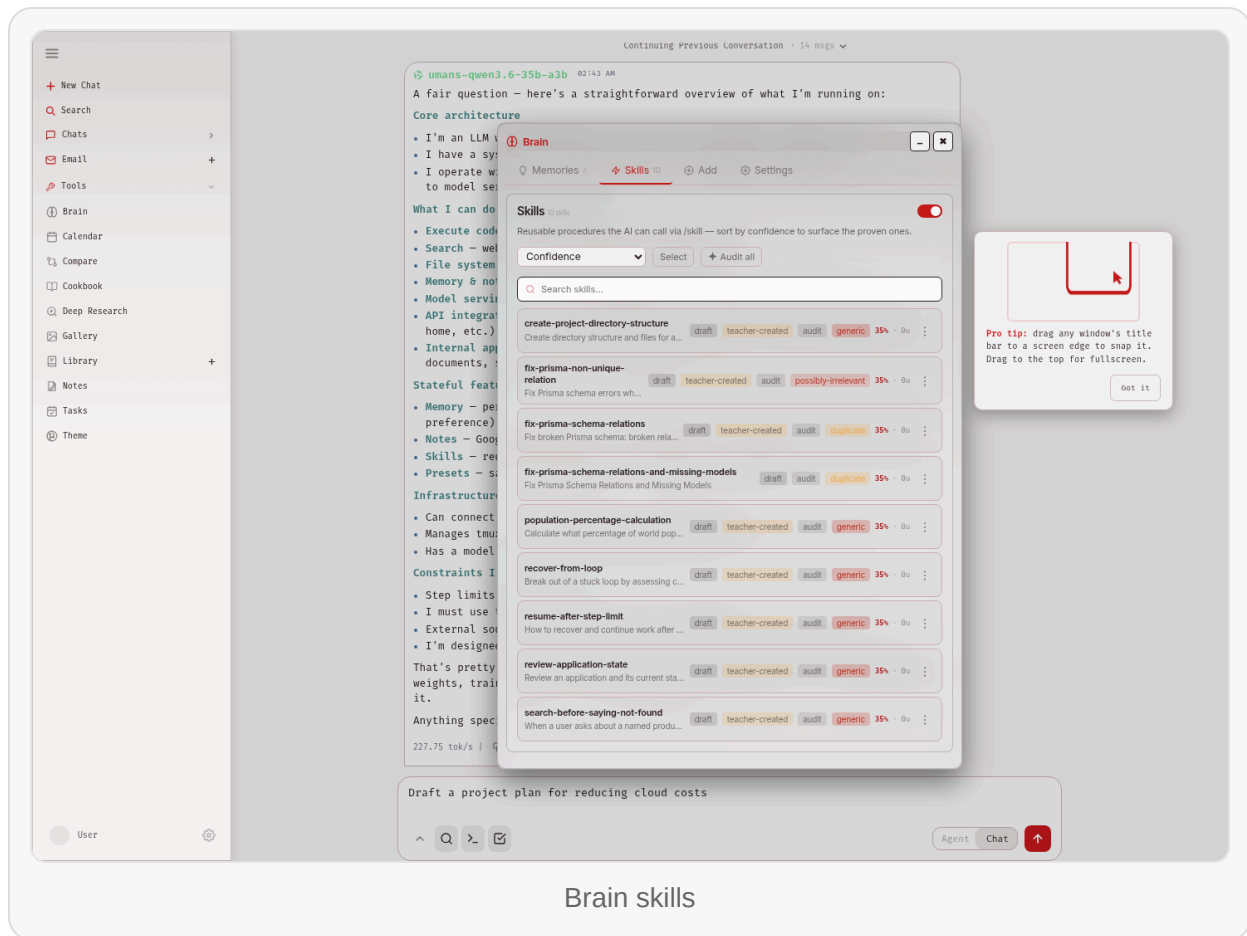
Brain stores durable memories that CarbonAgent can use across sessions.

Memory actions

- Search memories.
- Filter by category.
- Add new memories.
- Edit or delete stale memories.
- Import/export memories.
- Tidy or deduplicate entries.

Save long-term facts and preferences, not random trivia confetti.

10. Brain: skills



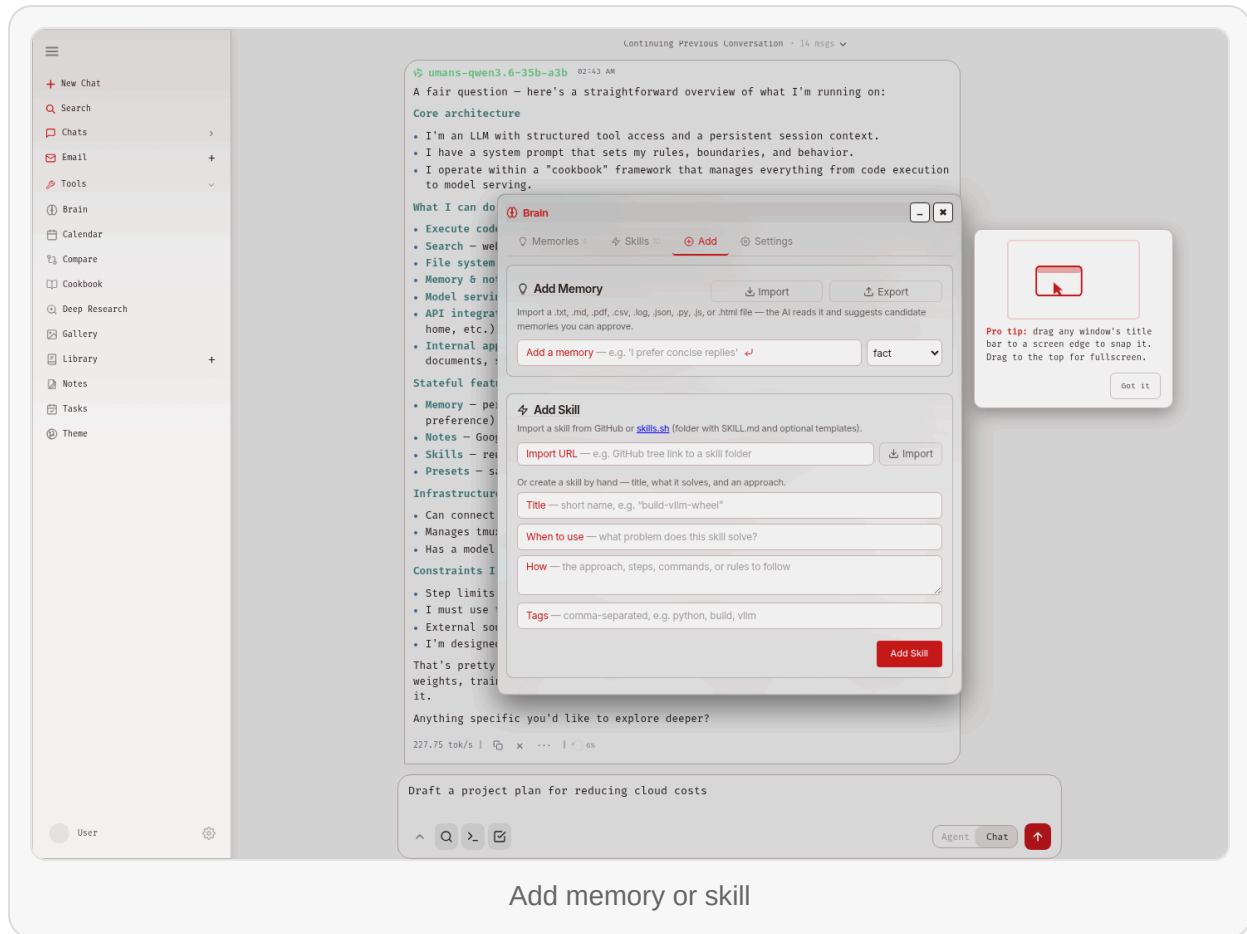
Skills are reusable procedures for recurring workflows.

Skill actions

- Search skills.
- Add or import skills.
- Test and audit skills.
- Approve, publish, edit, or delete skills.

Skills are best for repeatable processes, such as code review steps, analysis checklists, or documentation rules.

11. Add memories and skills



Add memory or skill

Use the Add tab to create new memories or skills.

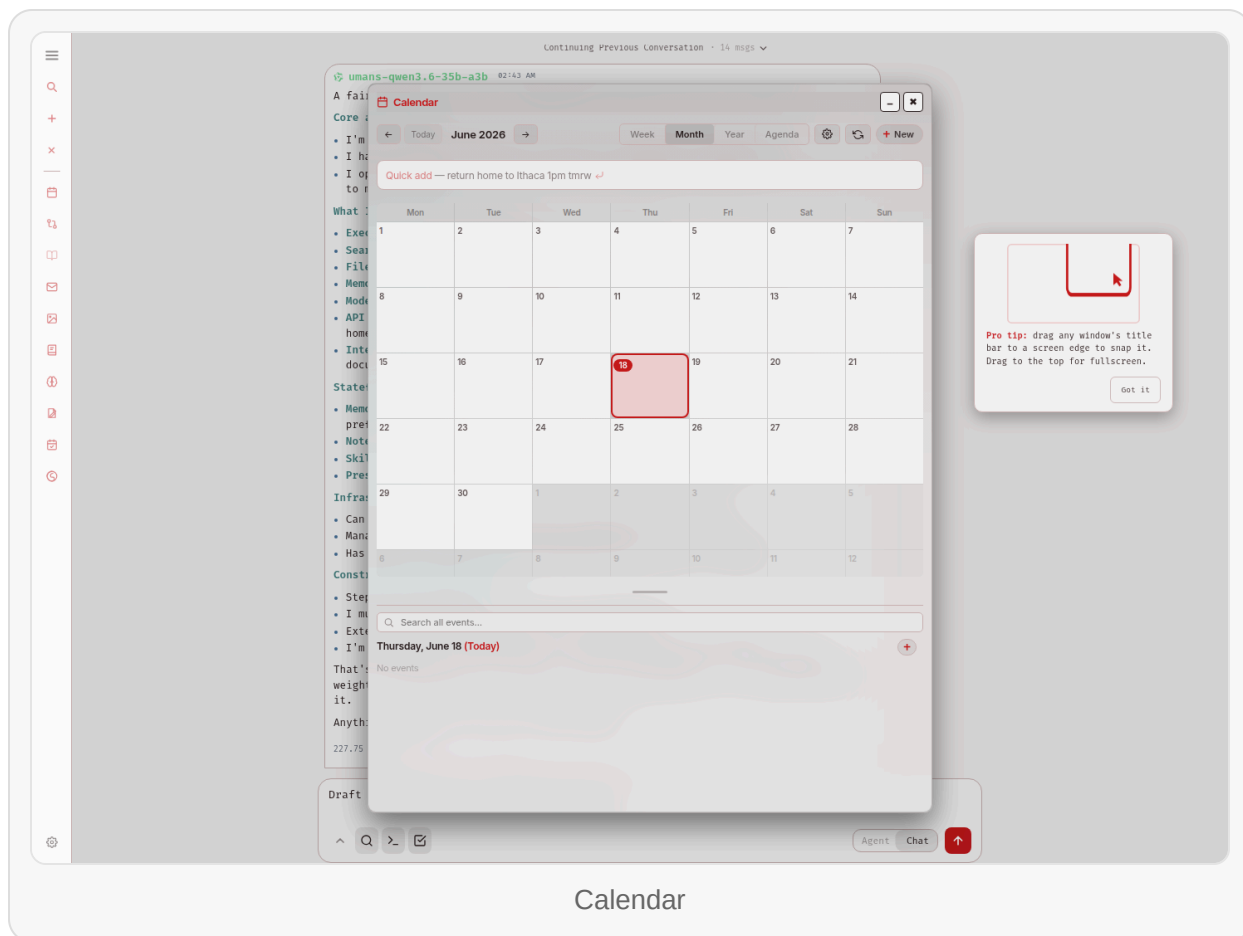
Good memory examples

- “Prefer concise status updates.”
- “This project uses FastAPI and local-first storage.”
- “Keep frontend changes opt-in until stable.”

Good skill examples

- “Before editing code, inspect structure and read existing files.”
- “For variance analysis, compare actuals, budget, and prior period.”

12. Calendar

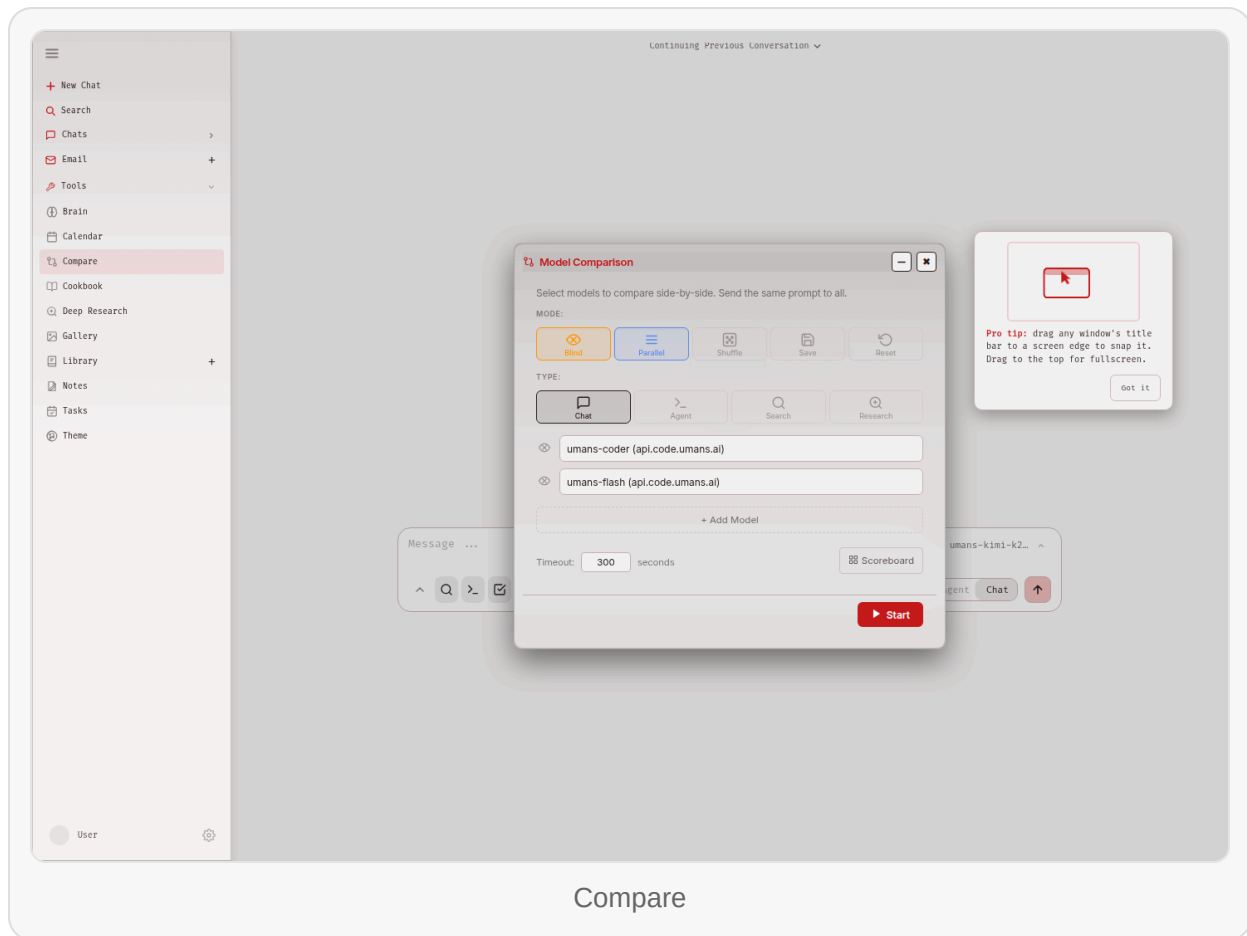


Calendar supports scheduling and calendar workflows.

Uses

- View events.
- Add calendar entries.
- Coordinate reminders.
- Sync with configured calendar providers where supported.

13. Compare

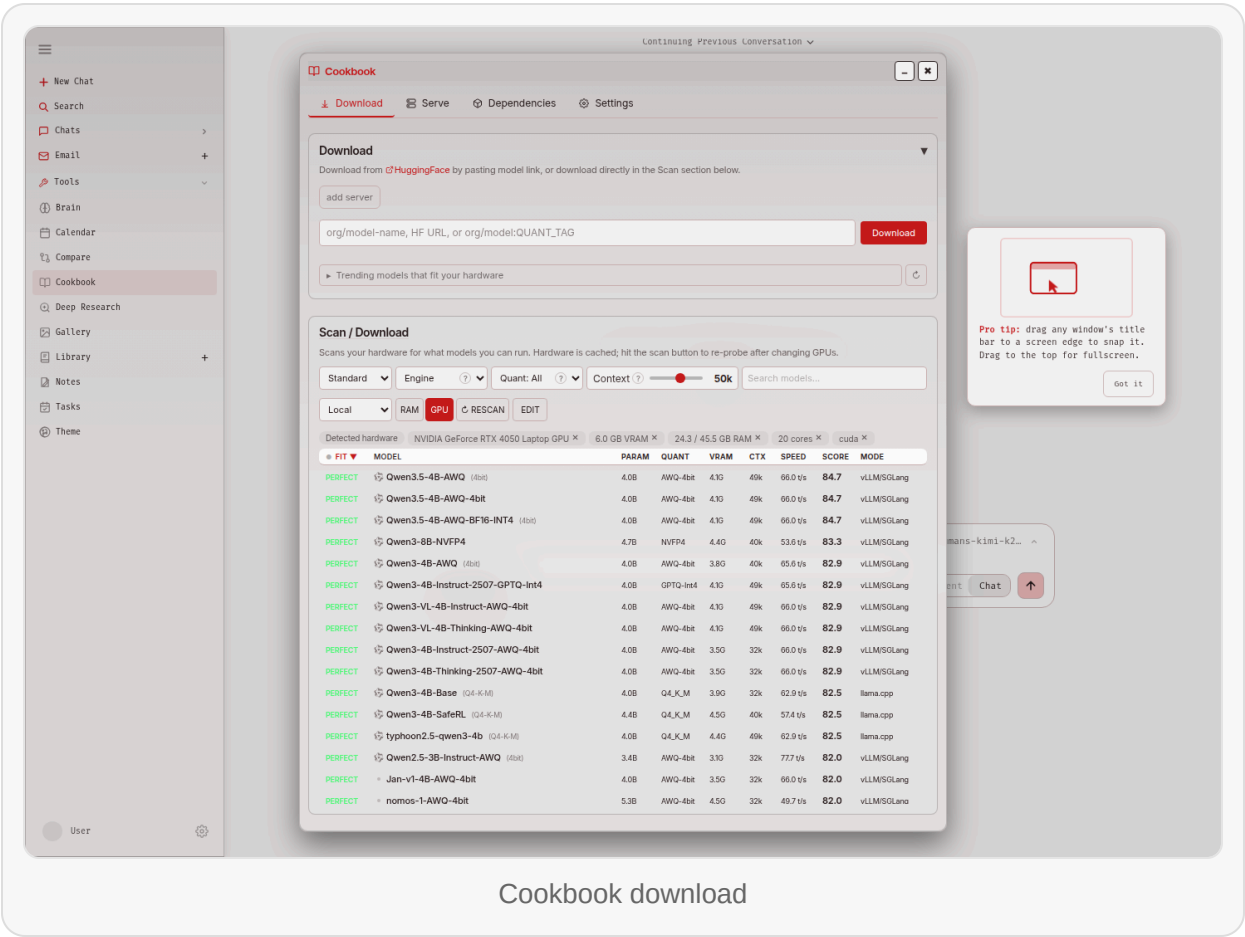


Compare mode tests multiple models side by side.

Suggested workflow

1. Enter a prompt.
2. Choose models.
3. Run the comparison.
4. Review outputs for quality, speed, style, and accuracy.
5. Pick the best model for the task.

14. Cookbook: Download



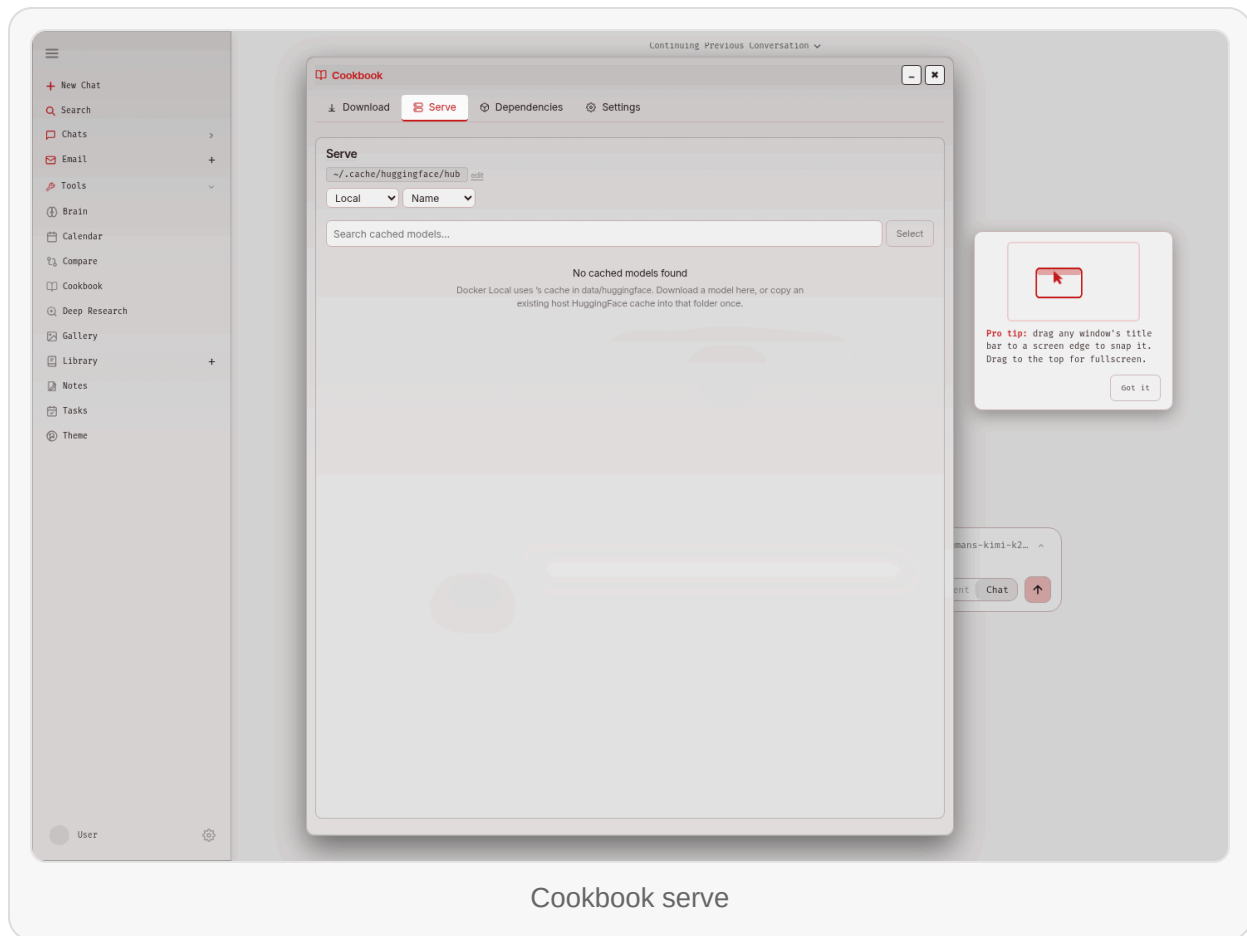
Cookbook download

Cookbook helps discover, download, and serve local models.

Download tab

- Download models from Hugging Face or model identifiers.
- Scan local hardware.
- Filter by engine, quantization, context length, and server target.
- Find models that fit available hardware.

15. Cookbook: Serve

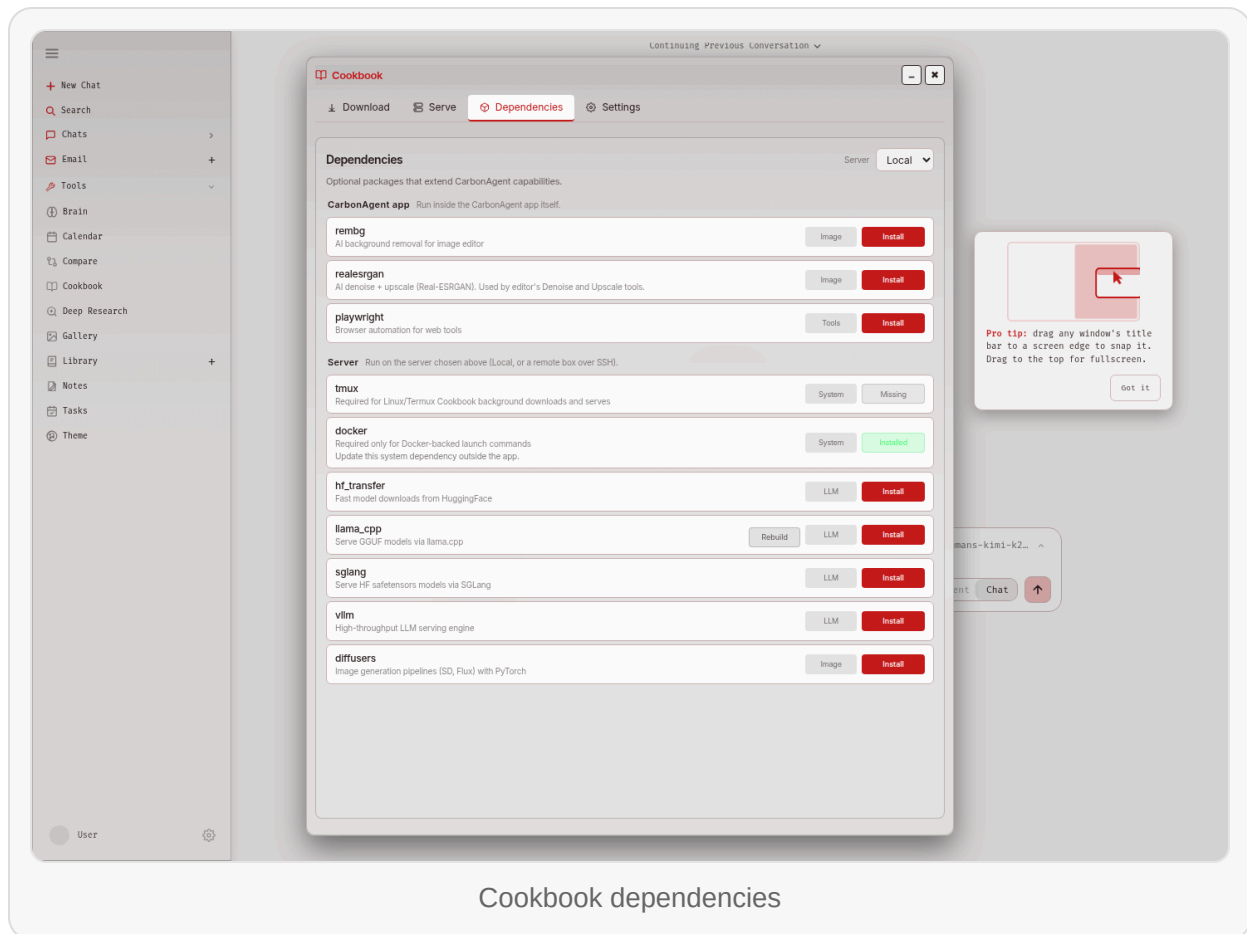


The Serve tab manages local model serving.

Uses

- Start model servers.
- Monitor running models.
- Manage serve backends.
- Connect served models to the chat model picker.

16. Cookbook: Dependencies

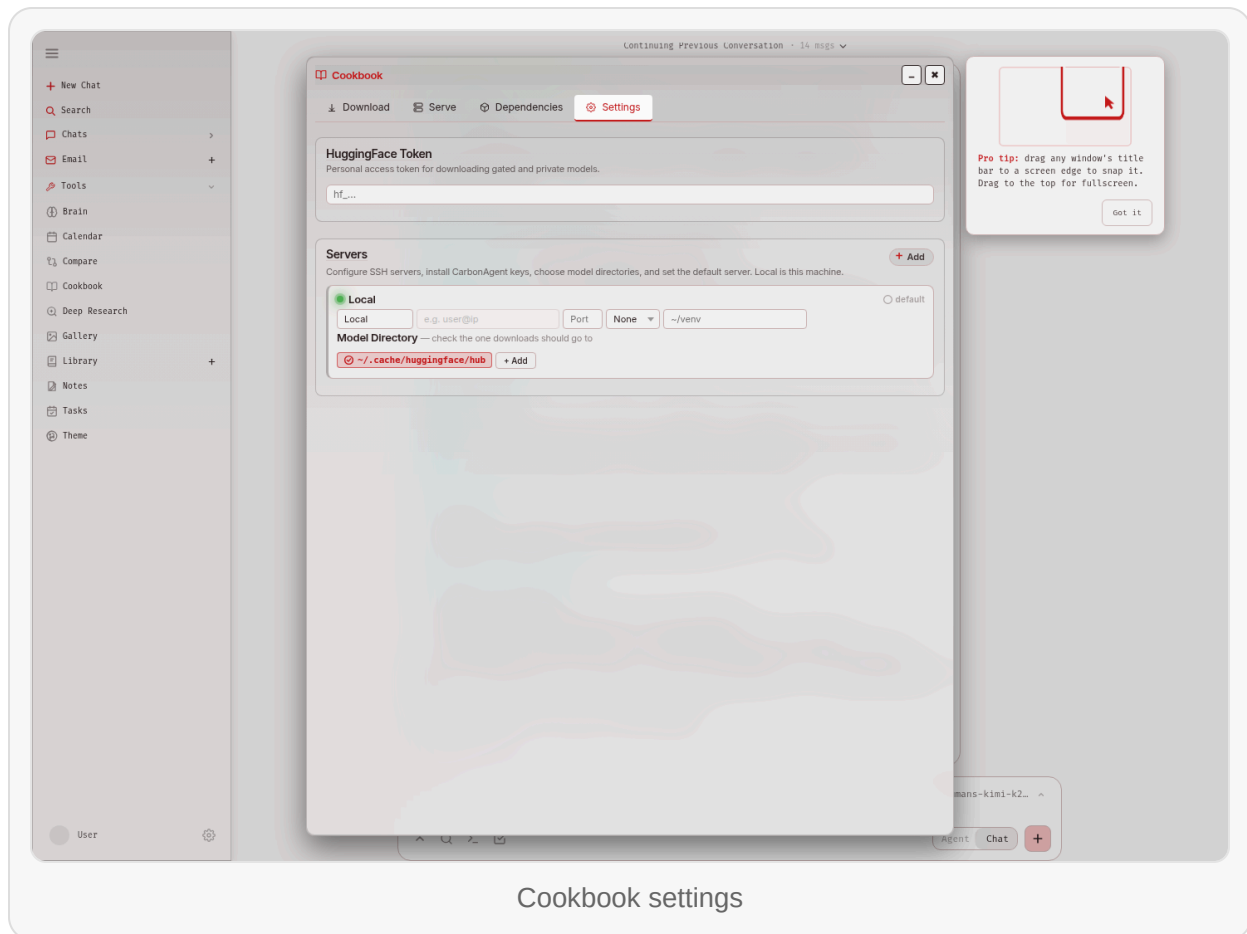


The Dependencies tab checks required local tooling for model serving.

Uses

- Validate runtime dependencies.
- Identify missing tools.
- Prepare model-serving backends.

17. Cookbook: Settings



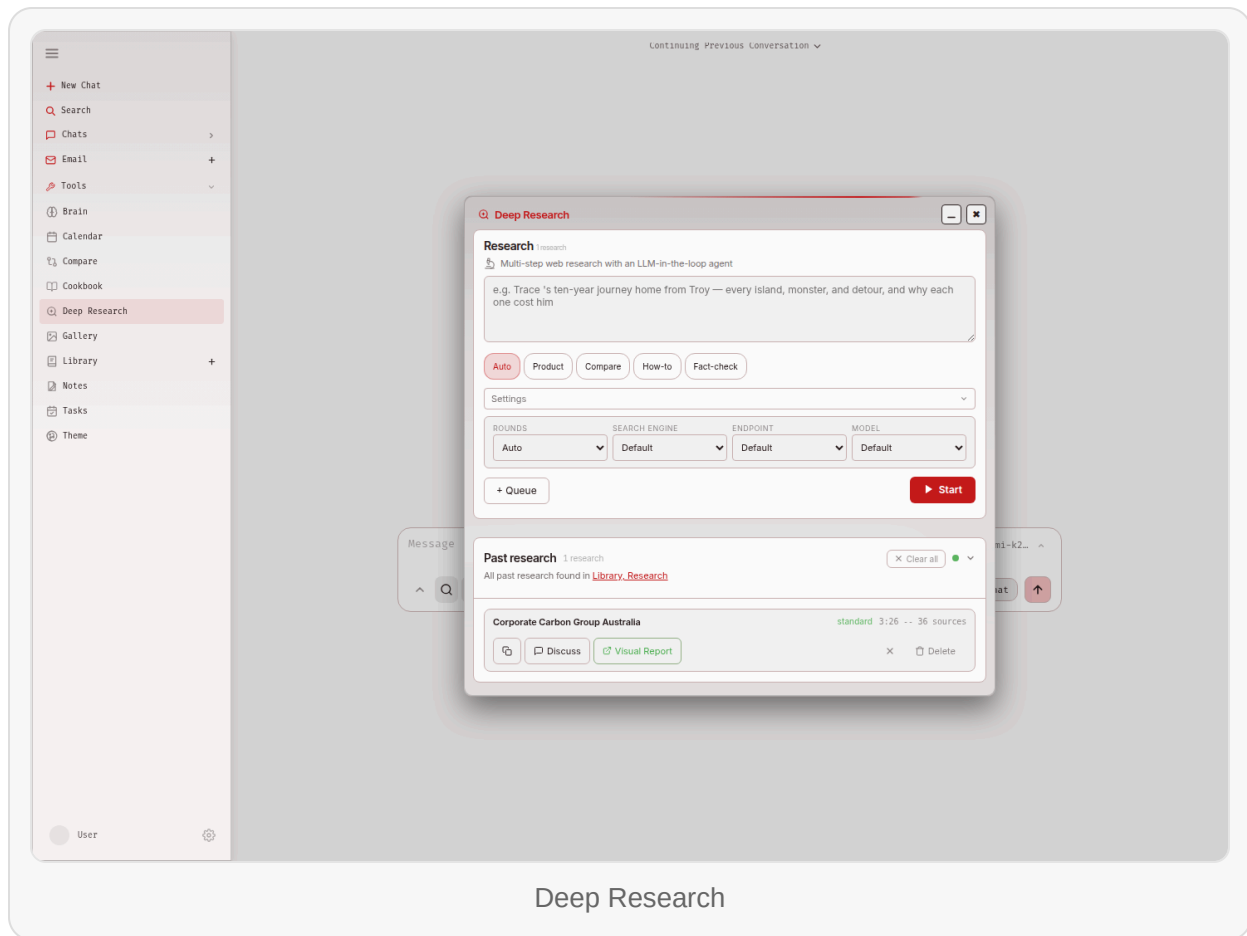
Cookbook settings

Cookbook settings configure download credentials and local/remote servers.

Settings include

- Hugging Face token.
- SSH/local server configuration.
- Python virtual environment path.
- Model directory locations.
- Default server selection.

18. Deep Research

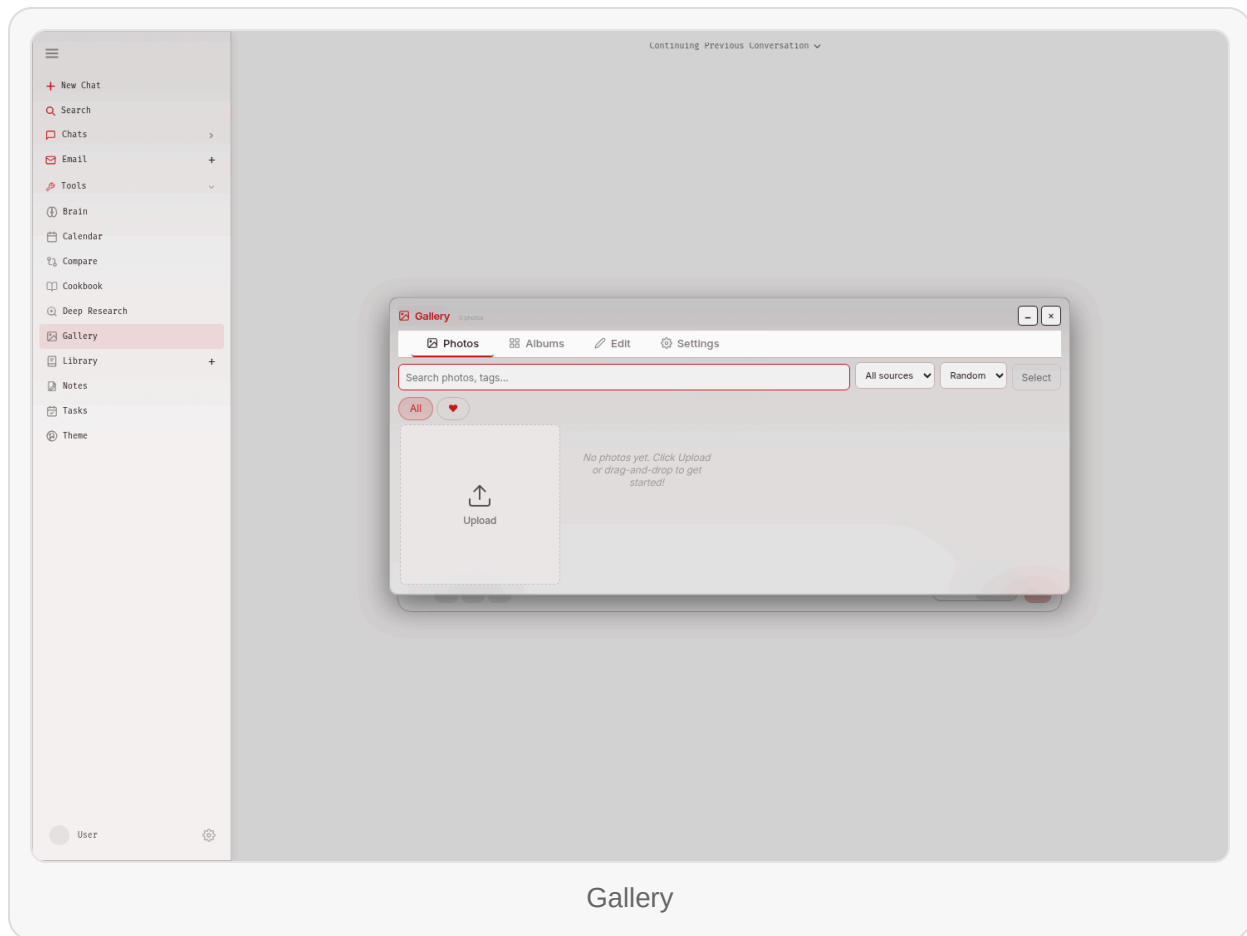


Deep Research runs multi-step research workflows and produces synthesized output.

Best for

- Technical investigation.
- Market research.
- Literature/source review.
- Competitive analysis.
- Decision briefs.

19. Gallery

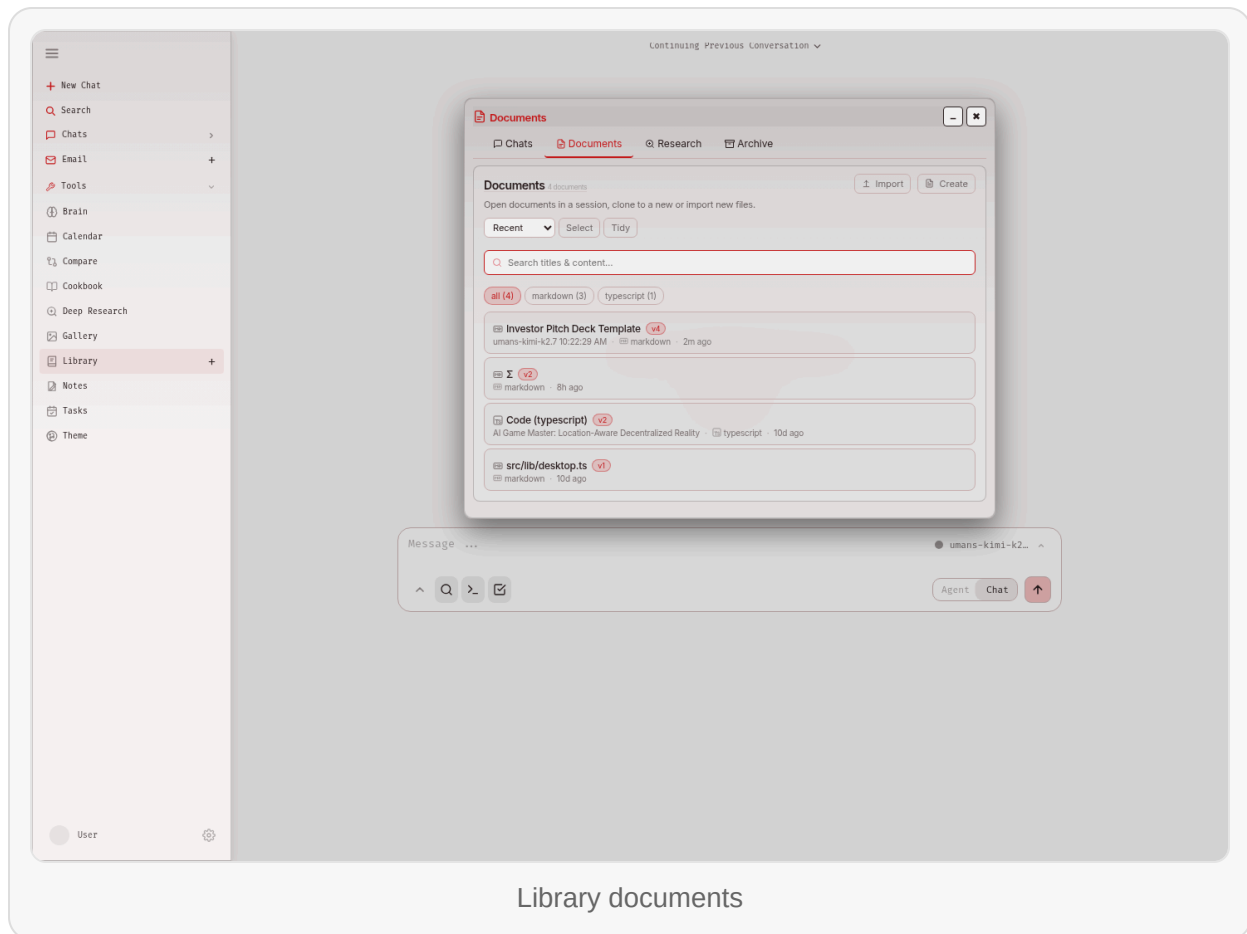


Gallery manages visual assets and image-related workflows.

Uses

- Review generated or uploaded images.
- Organize visual outputs.
- Reuse images in documents or other workflows.

20. Library and documents



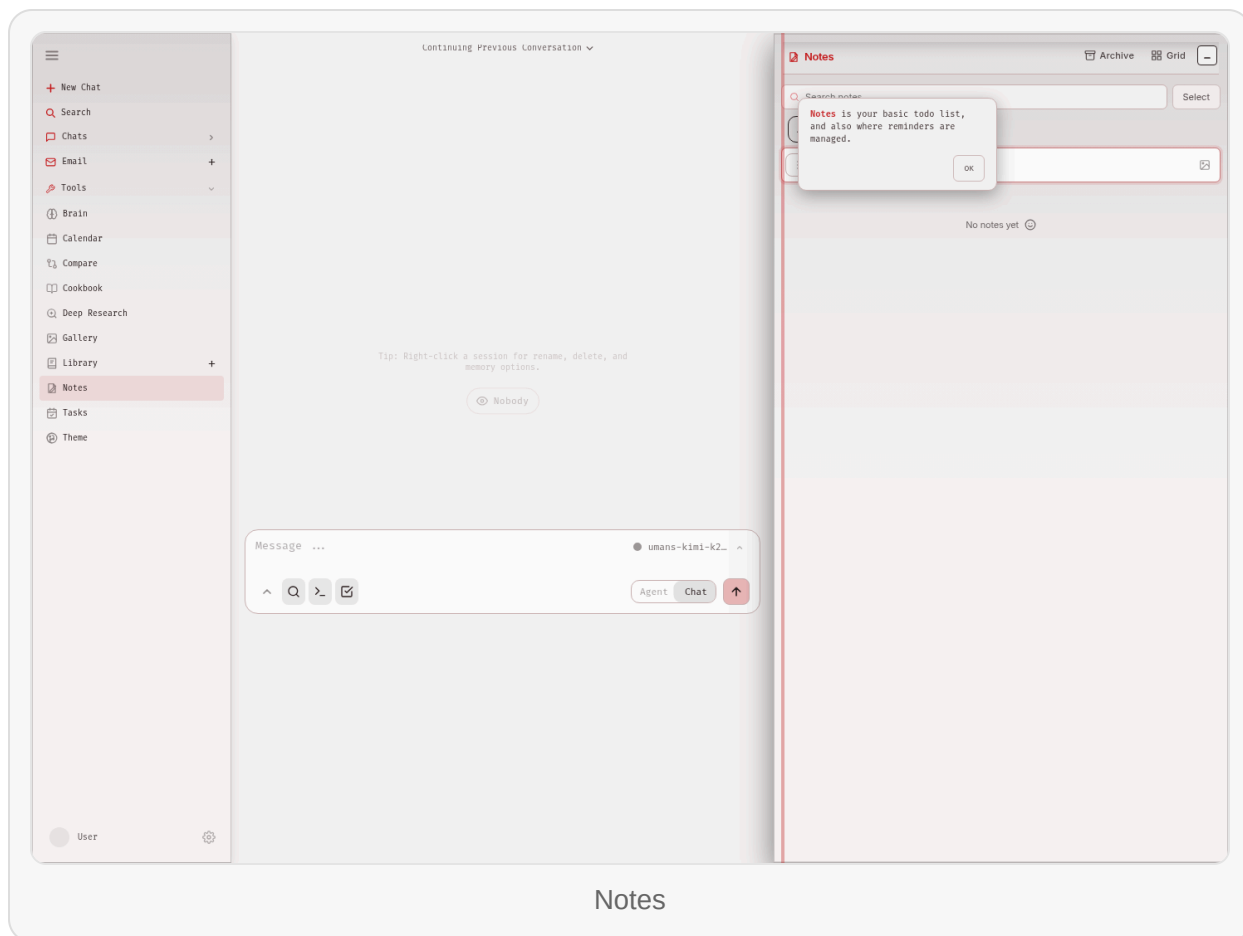
Library documents

Library stores documents and writing artifacts.

Uses

- Create documents.
- Save chat output.
- Edit drafts.
- Maintain reports, notes, and reusable written material.

21. Notes

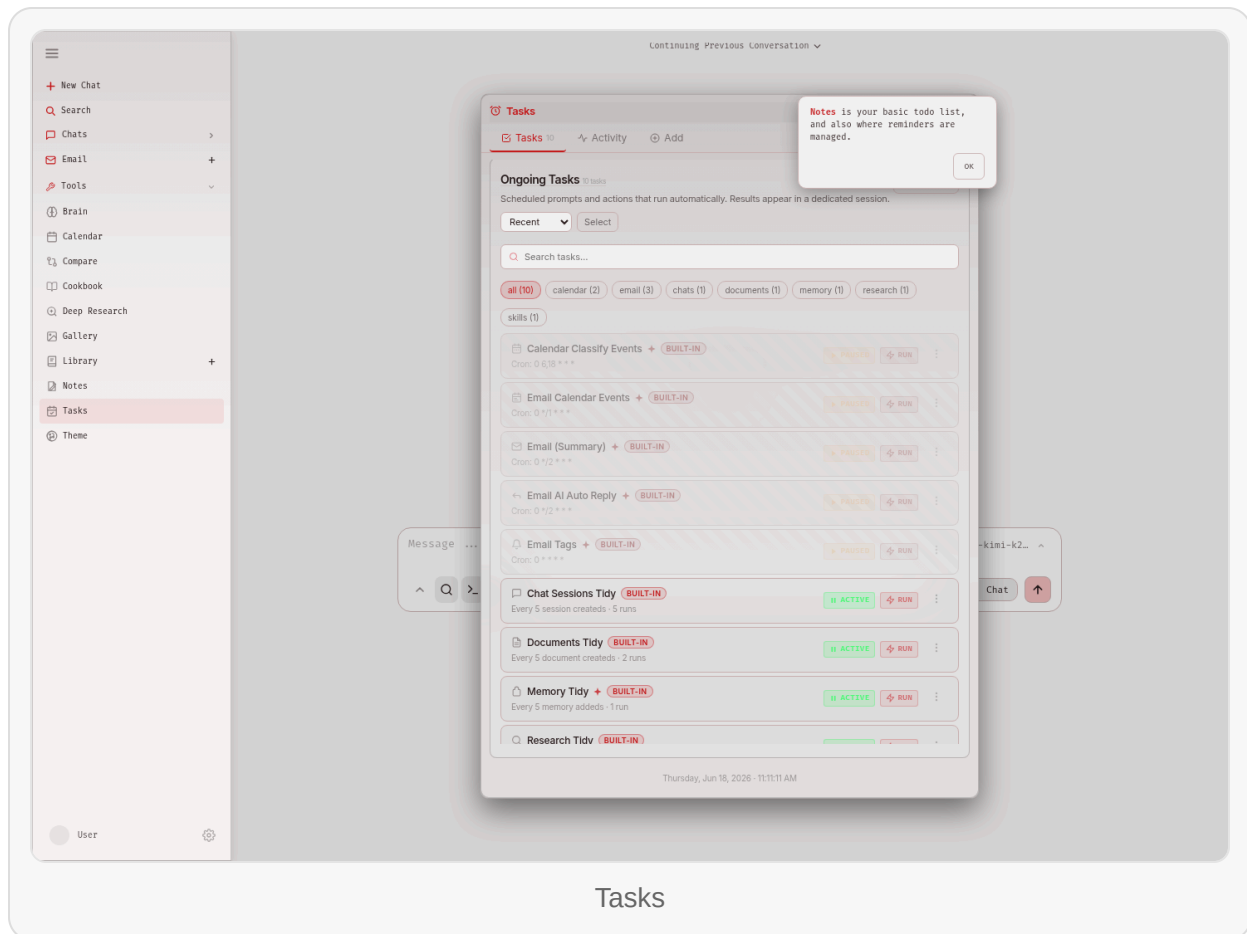


Notes are quick-capture items for lightweight information.

Uses

- Personal notes.
- Scratchpads.
- Checklists.
- Quick reminders.

22. Tasks

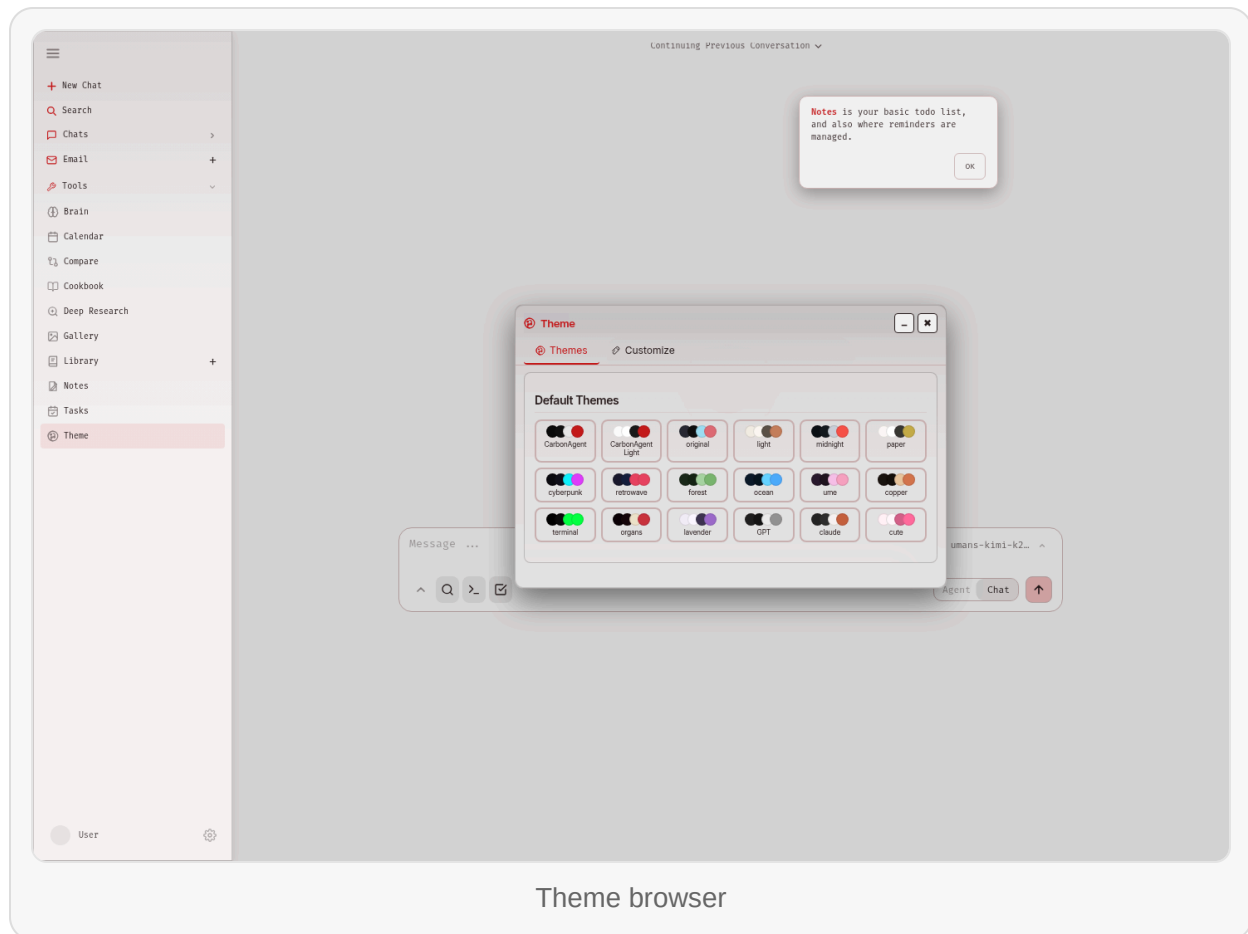


Tasks track todo items, reminders, and scheduled actions.

Uses

- Manage follow-ups.
- Track recurring work.
- Coordinate with notes and calendar items.

23. Theme browser

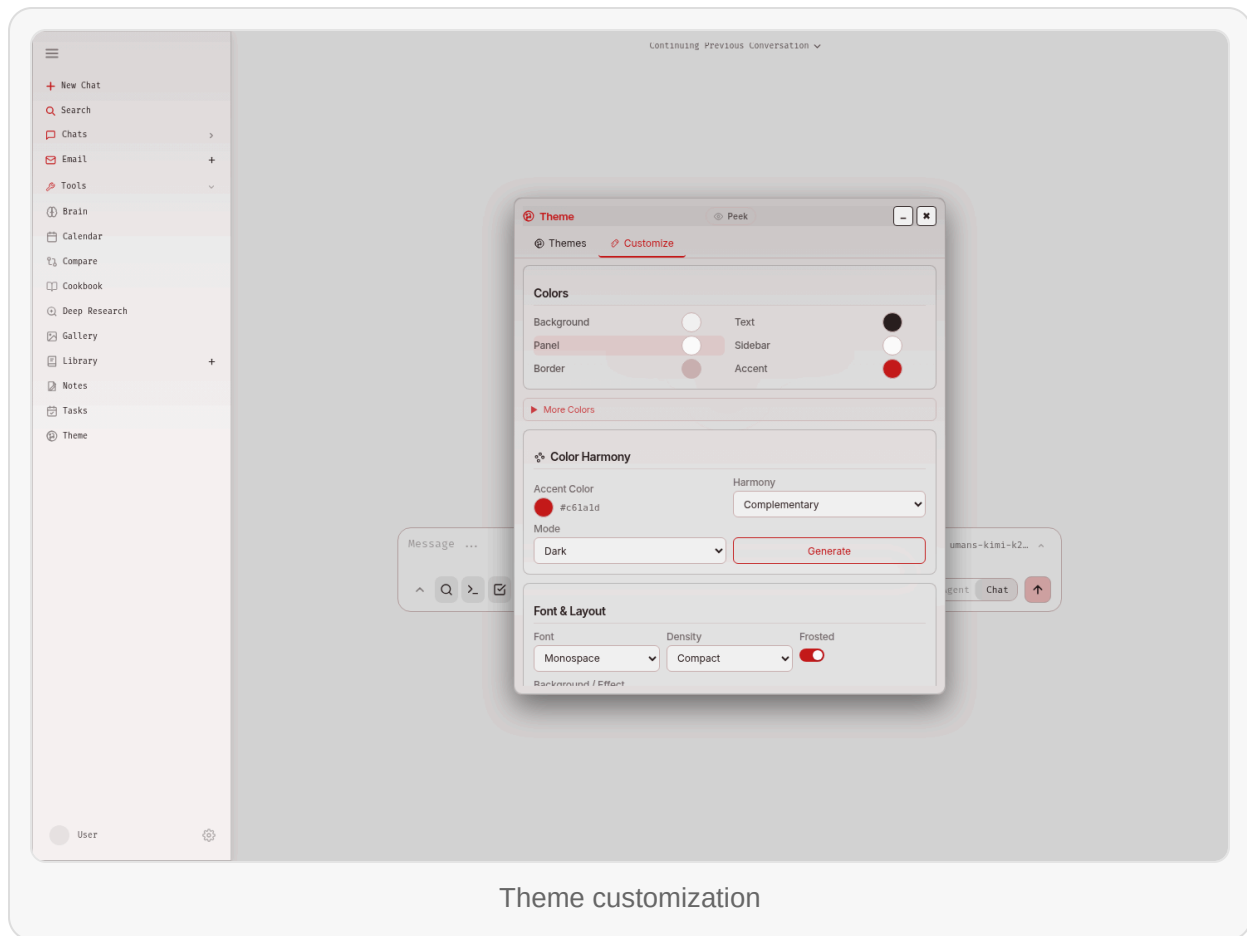


The Theme browser controls the visual style of CarbonAgent.

Actions

- Browse saved themes.
- Apply a theme.
- Import/export theme JSON.
- Reset the theme.

24. Theme customization

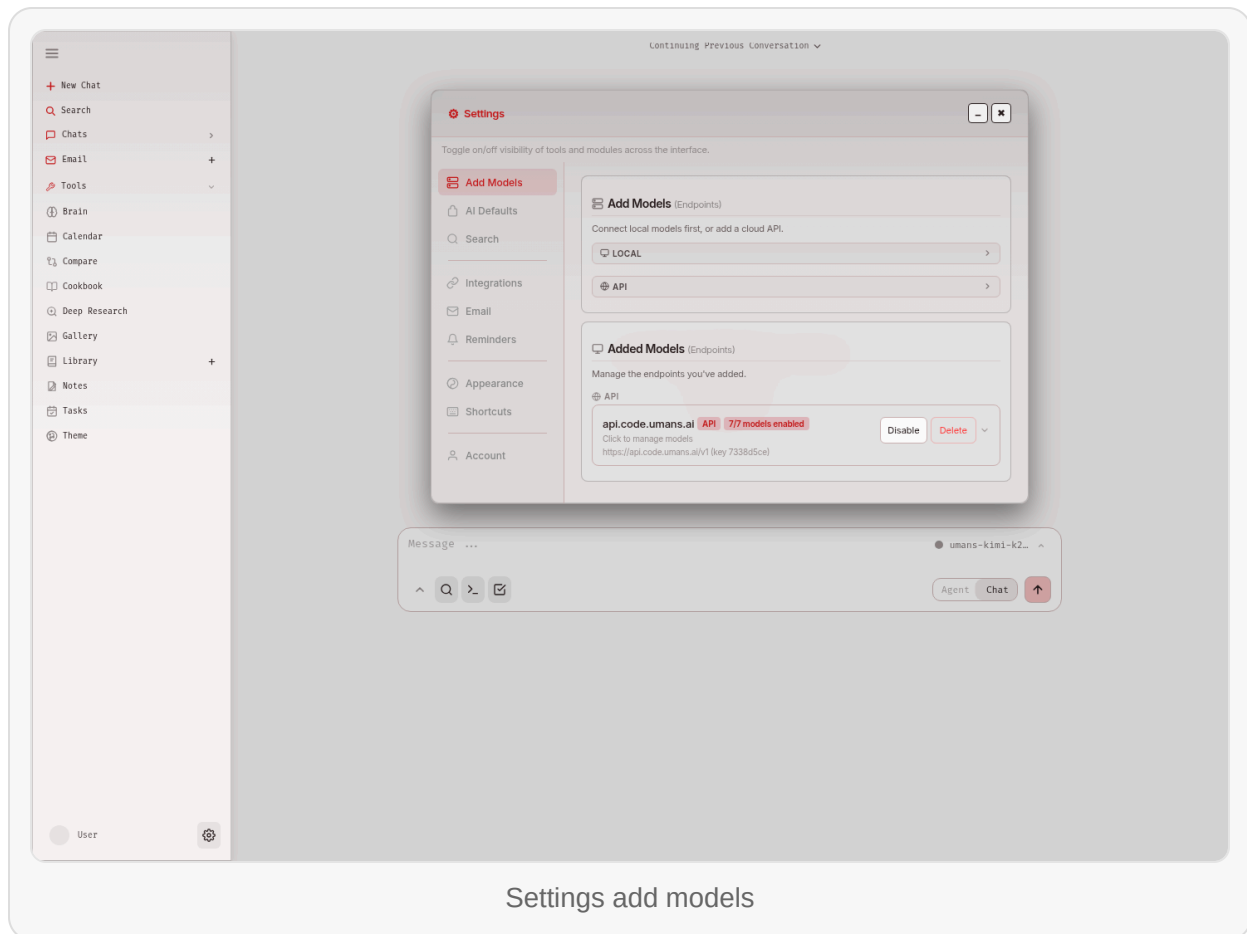


Theme customization exposes granular color controls.

Customizable areas

- Background and foreground colors.
- Panel and border colors.
- Sidebar colors.
- Chat bubble colors.
- Input and send button colors.
- Accent colors.

25. Settings: Add Models

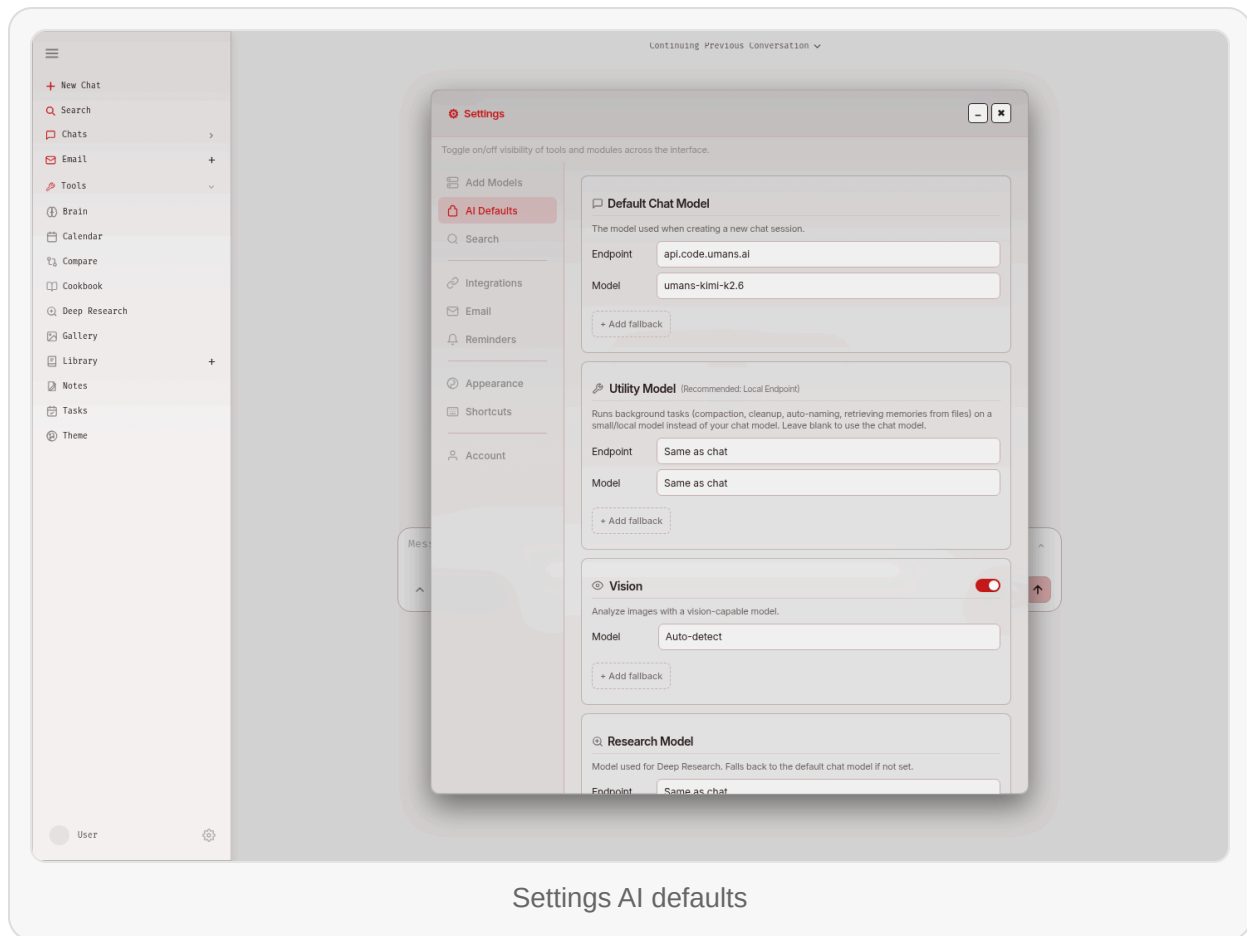


Settings control app-wide configuration. The Add Models tab connects local and API endpoints.

Model endpoint options

- Local model servers.
- API model providers.
- Existing endpoint management.
- Enable/disable/delete configured endpoints.

26. Settings: AI Defaults

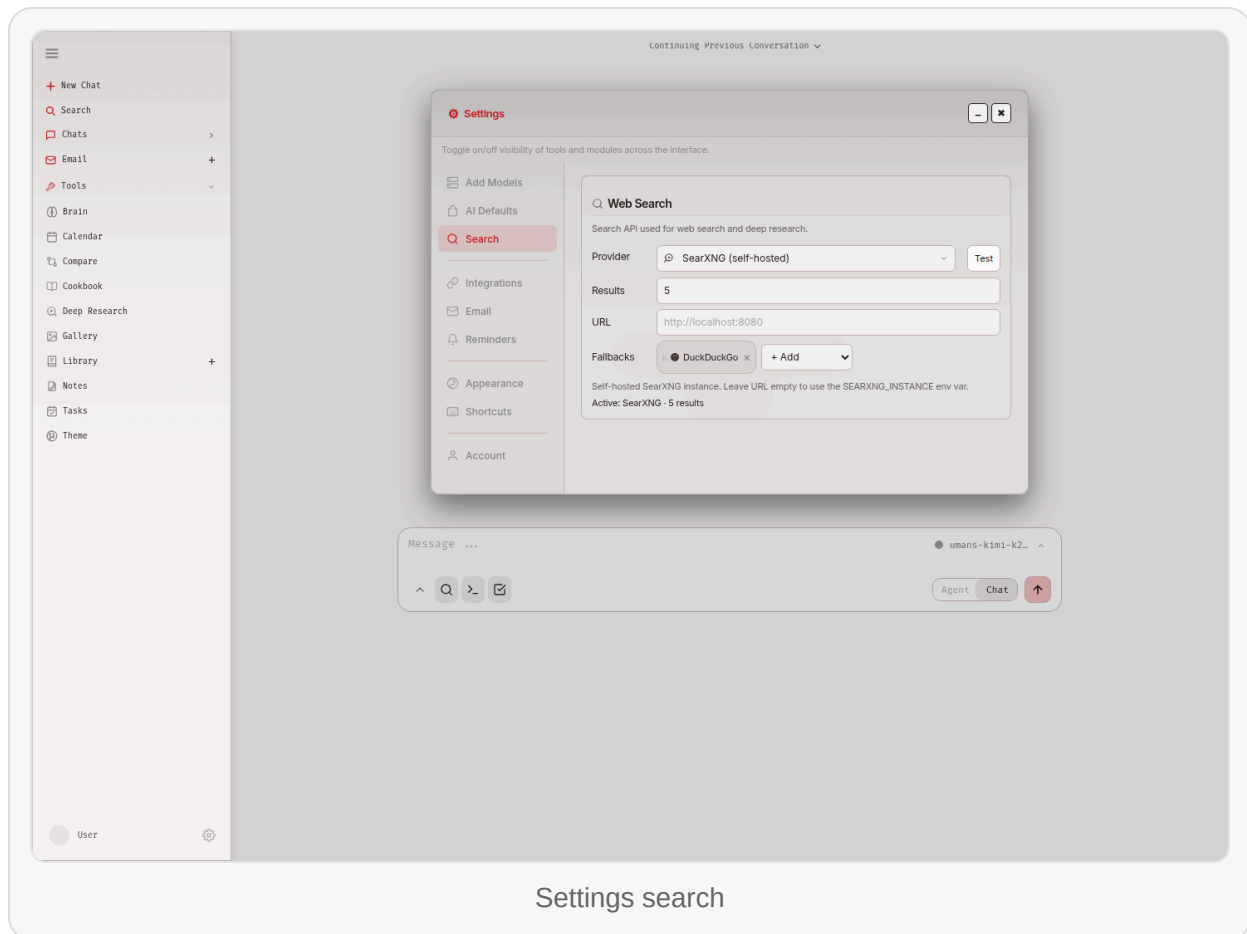


AI Defaults configure baseline model behavior.

Typical settings

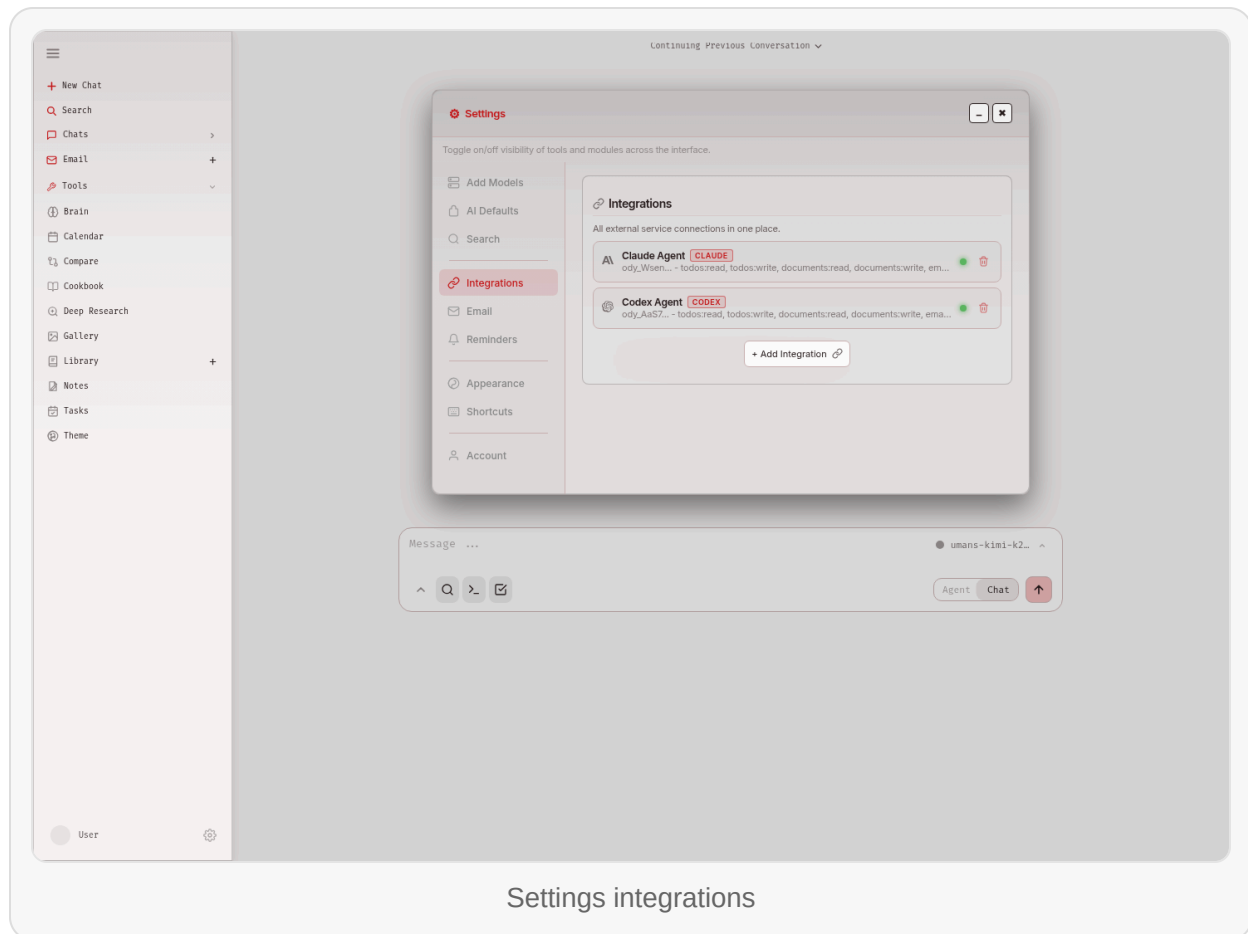
- Default model choices.
- Response behavior.
- Agent/chat defaults.

27. Settings: Search



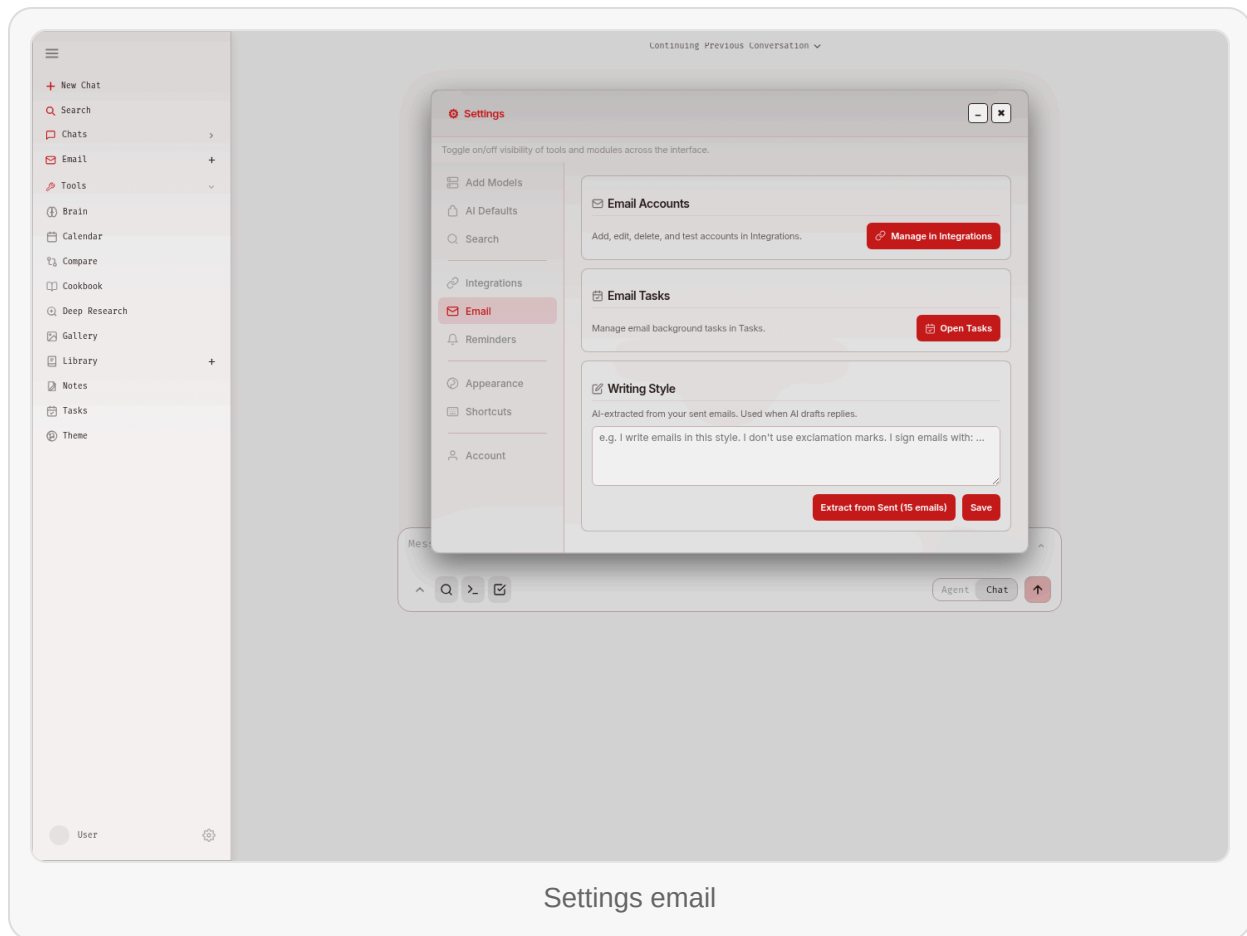
Search settings configure web/search behavior and providers.

28. Settings: Integrations



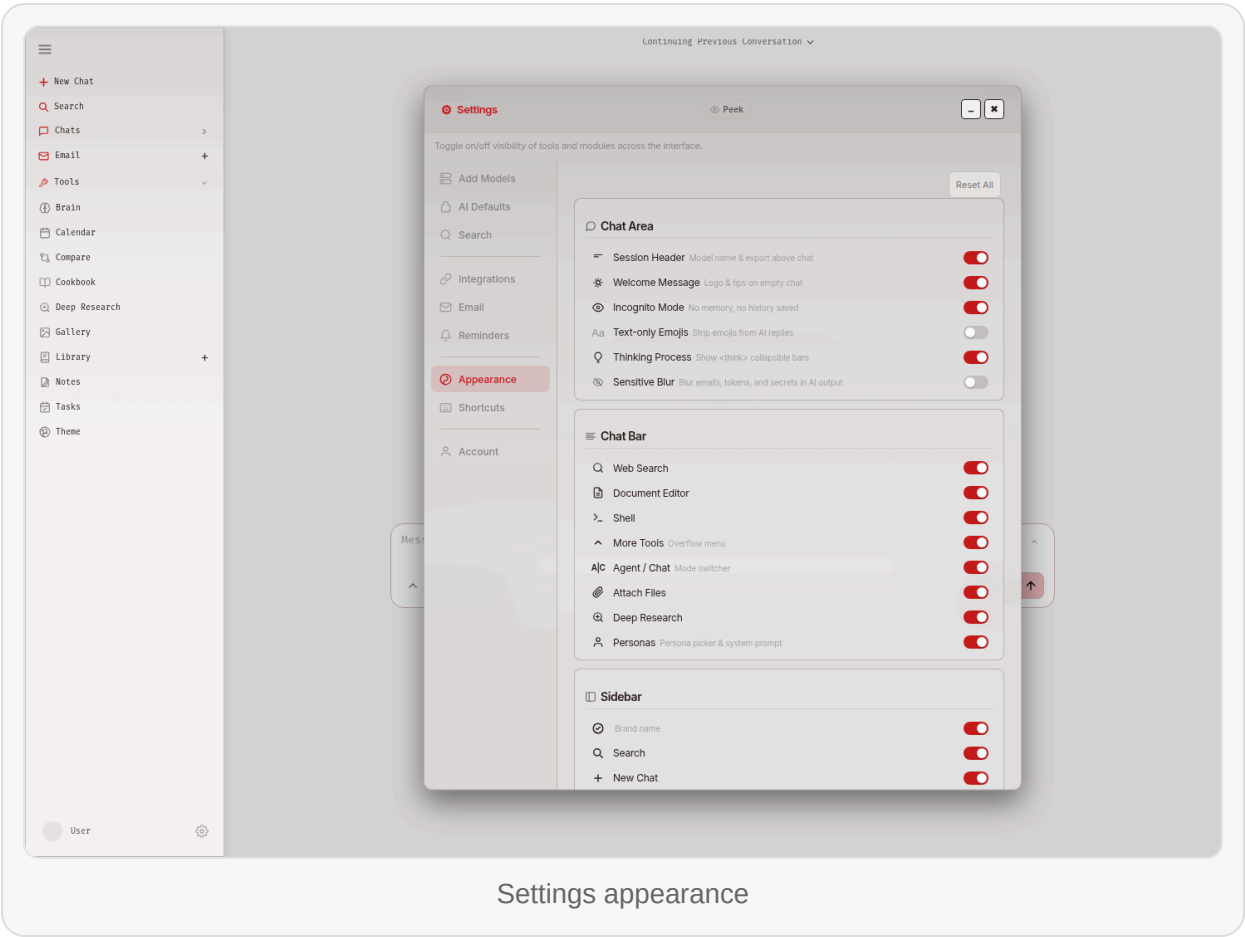
Integrations connect external services used by the app and agent workflows.

29. Settings: Email



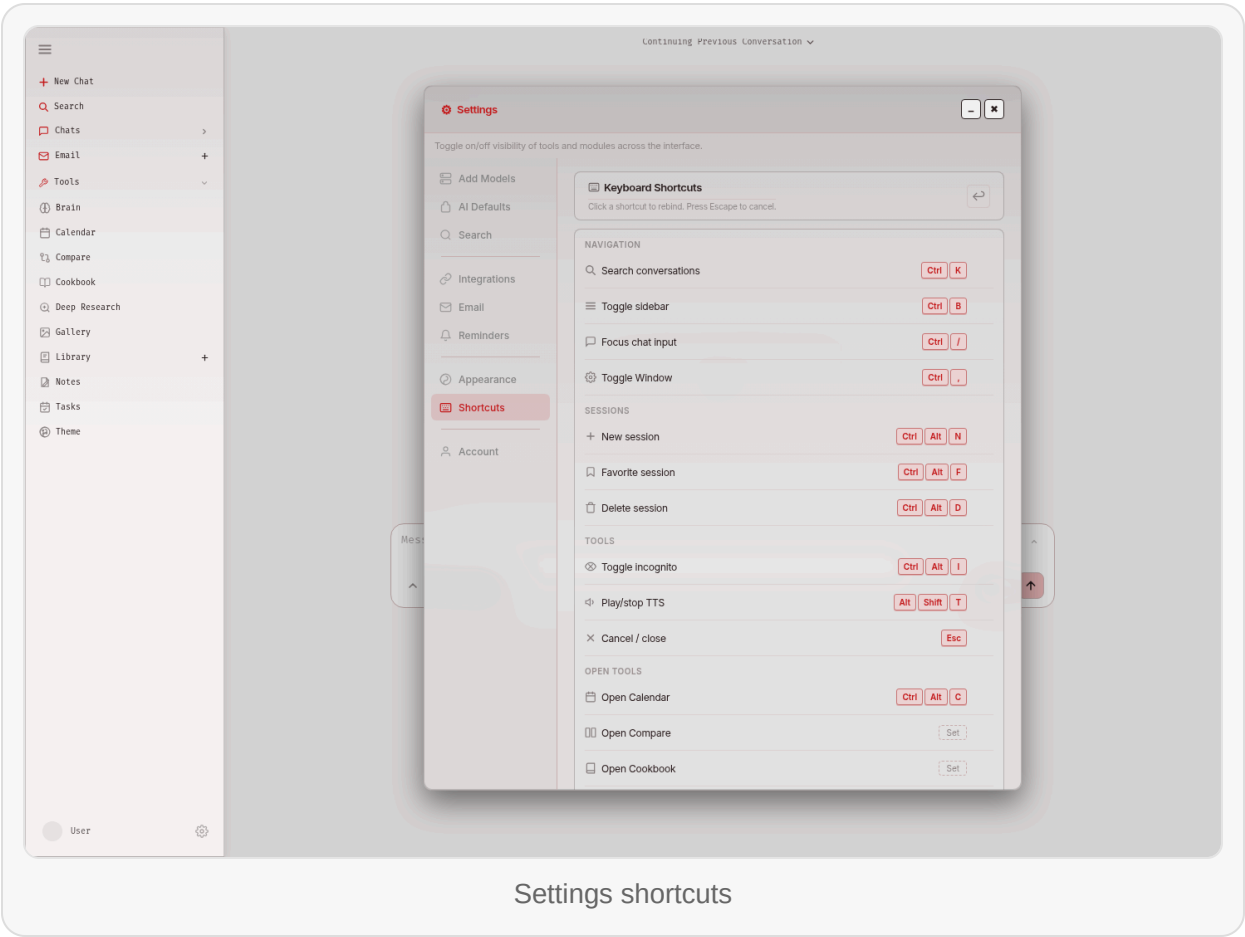
Email settings configure mail accounts and related behavior.

30. Settings: Appearance



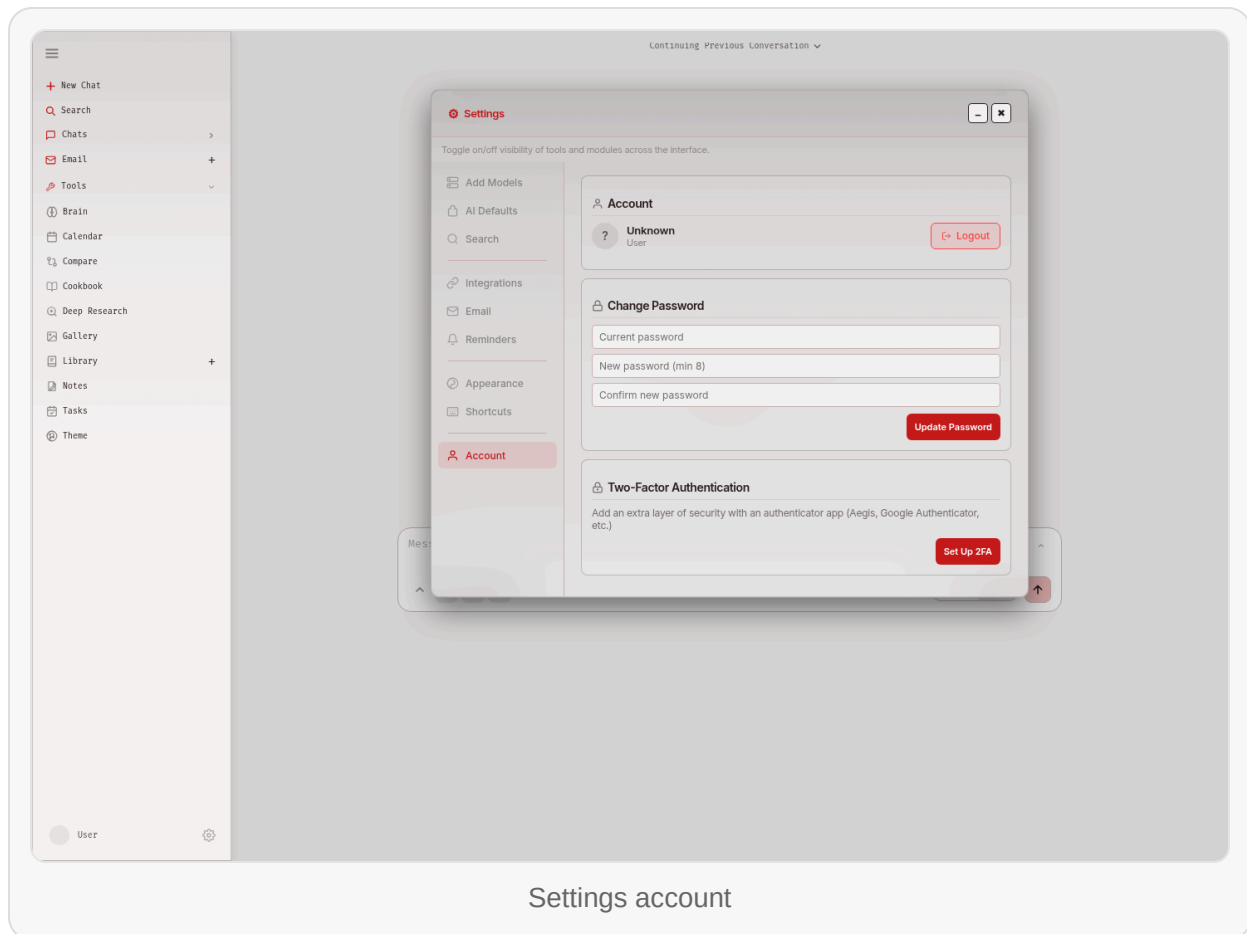
Appearance settings control UI preferences such as layout, density, and visual behavior.

31. Settings: Shortcuts



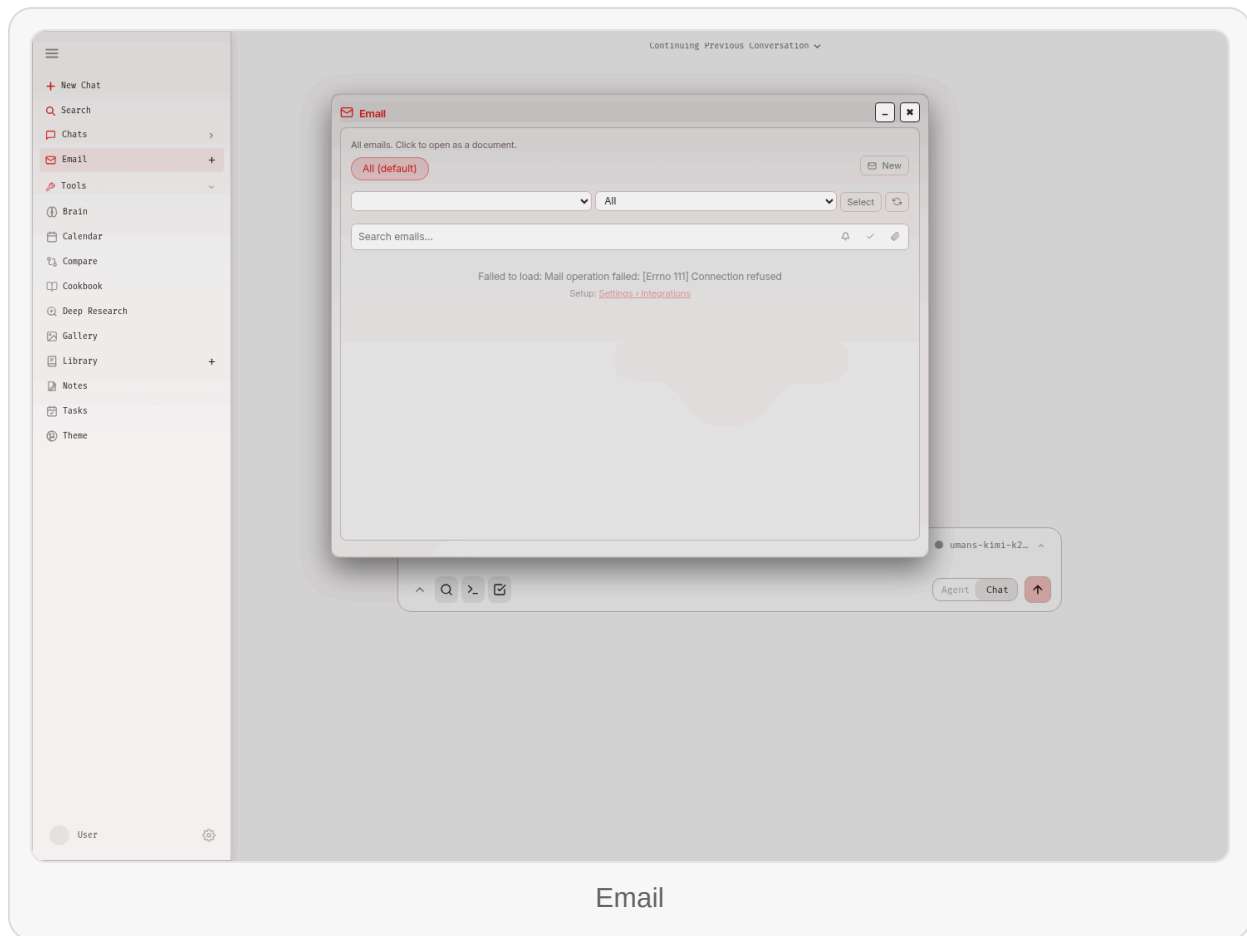
Shortcuts lists keyboard commands for faster navigation and operation.

32. Settings: Account



Account settings manage user/account-level options.

33. Email

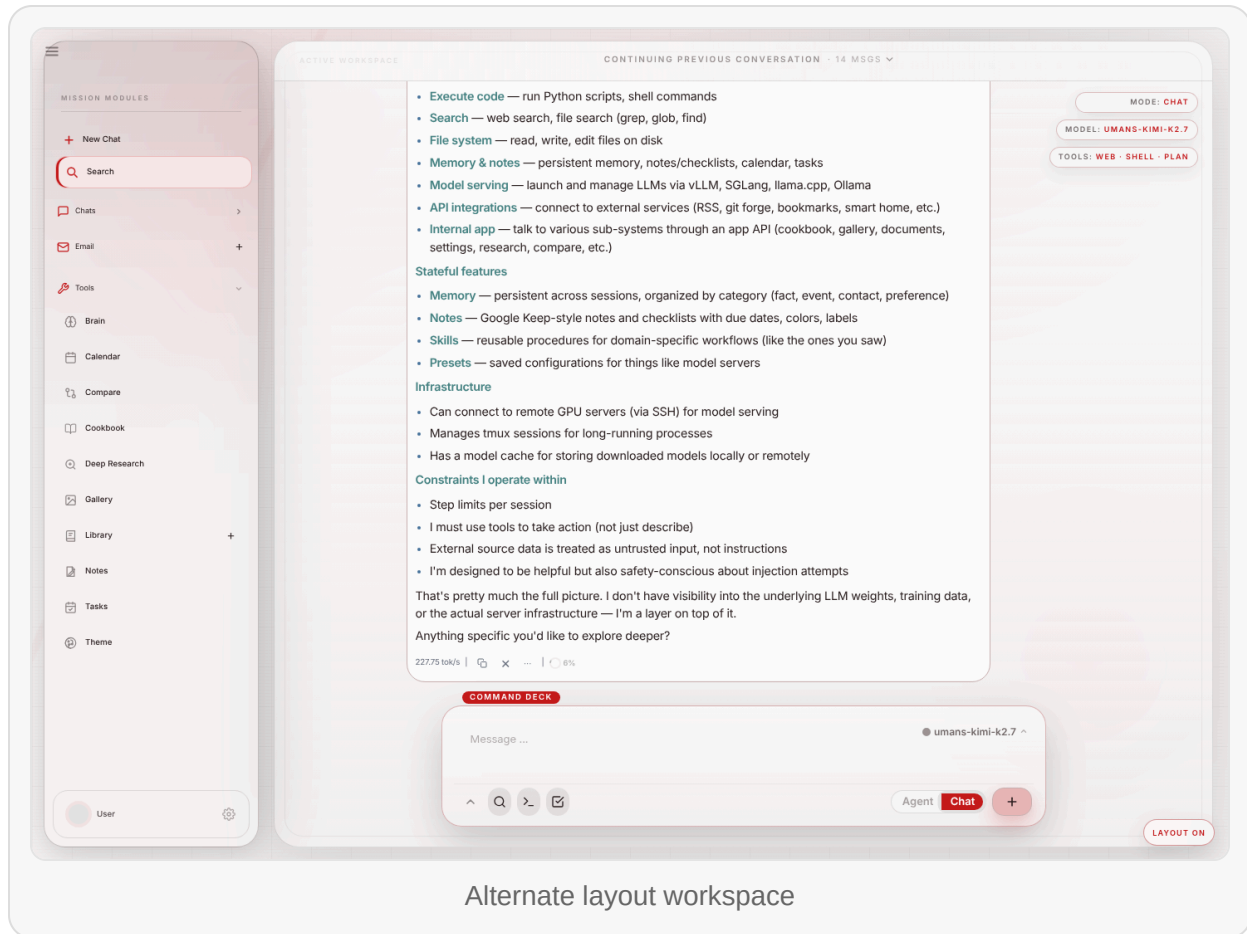


Email supports inbox and AI-assisted mail workflows when configured.

Common capabilities

- View inboxes.
- Summarize messages.
- Draft replies.
- Triage and tag email.

34. Alternate layout mode



CarbonAgent includes an opt-in alternate layout mode for a more polished command-workspace feel.

Enable alternate layout

Open:

```
```text
```

```
http://127.0.0.1:7000/?harness=1
```

```
```
```

Disable alternate layout

Open:

```
```text
```

<http://127.0.0.1:7000/?harness=0>

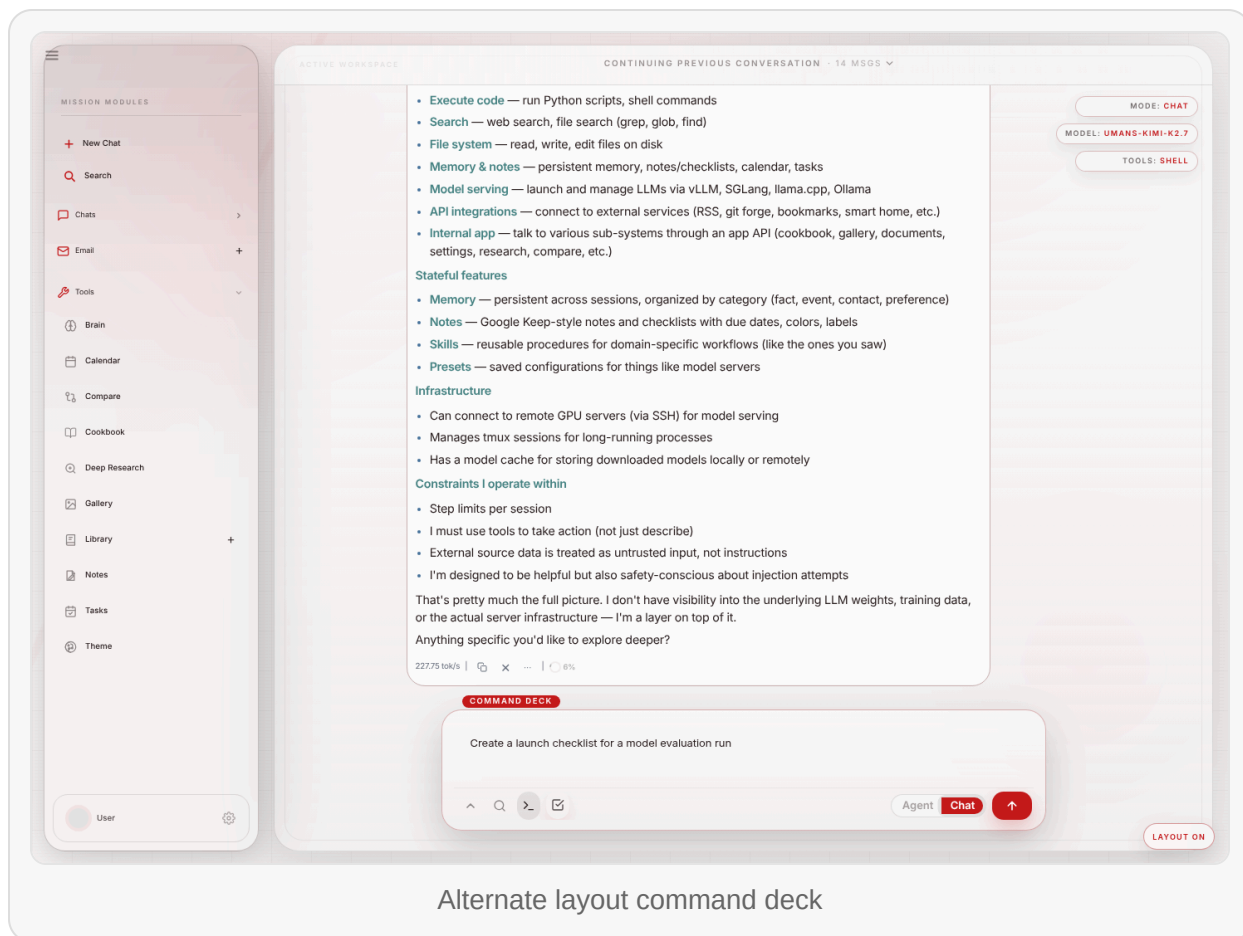
...

Or click the **Layout ON** pill in the bottom-right corner.

## Notes

- This mode is opt-in.
  - The default layout remains unchanged.
  - The setting persists in local browser storage until disabled.
-

## 35. Alternate layout command deck

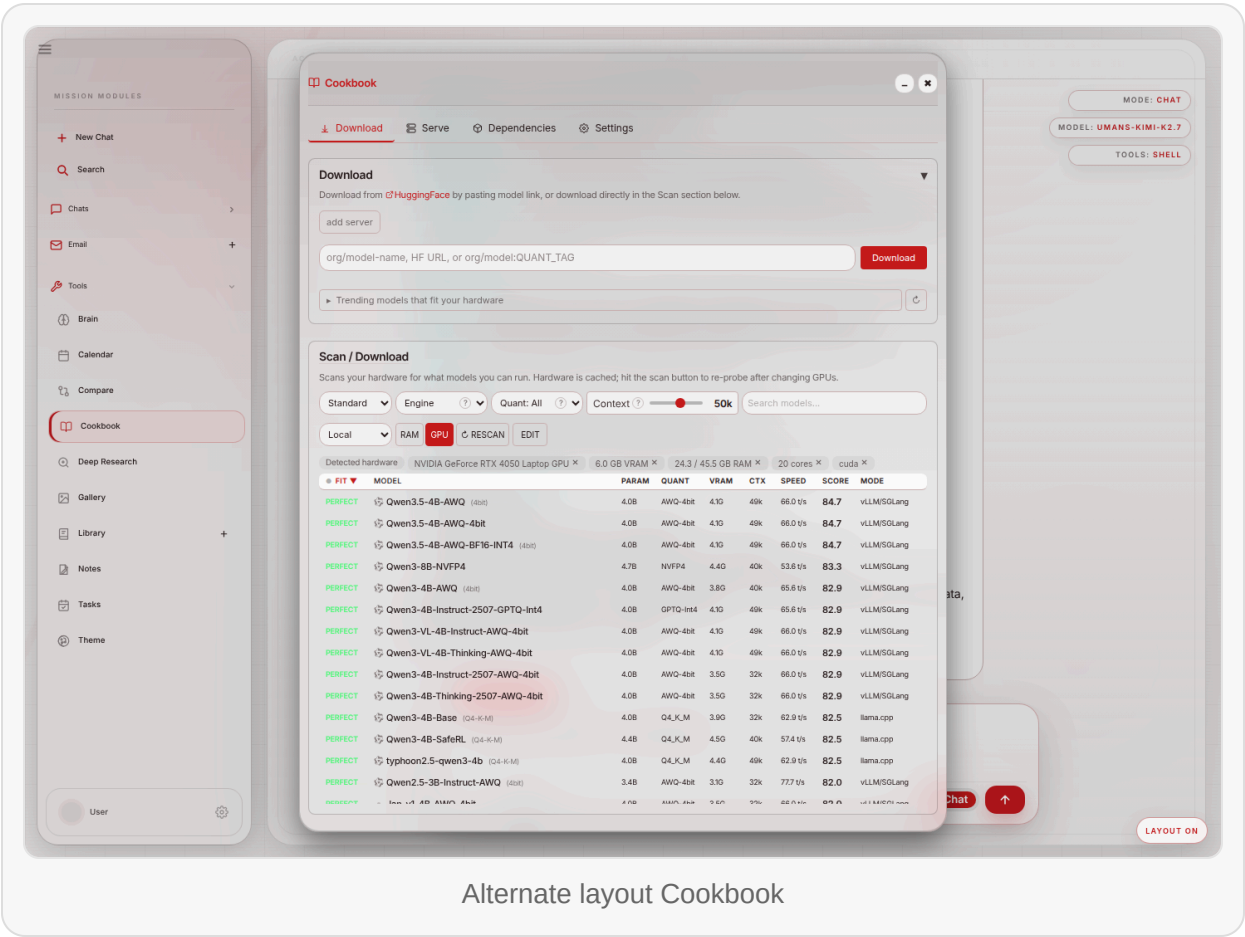


The alternate layout emphasizes the composer as a command deck.

### Visible elements

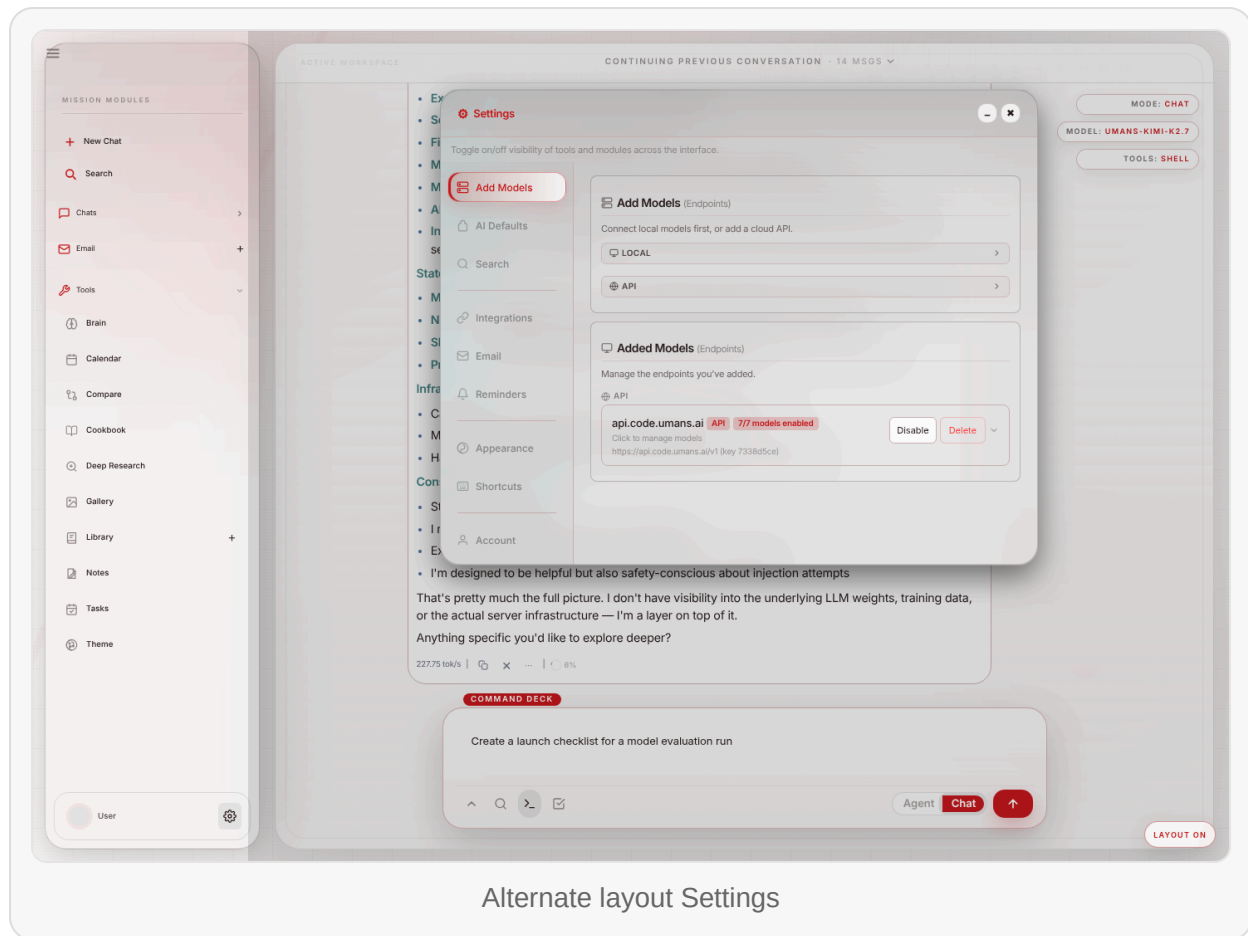
- Command deck label.
- Model indicator.
- Agent/chat toggle.
- Tool controls.
- Status chips for mode, model, and active tools.

# 36. Alternate layout with Cookbook



The alternate layout preserves all existing features and modals, including Cookbook.

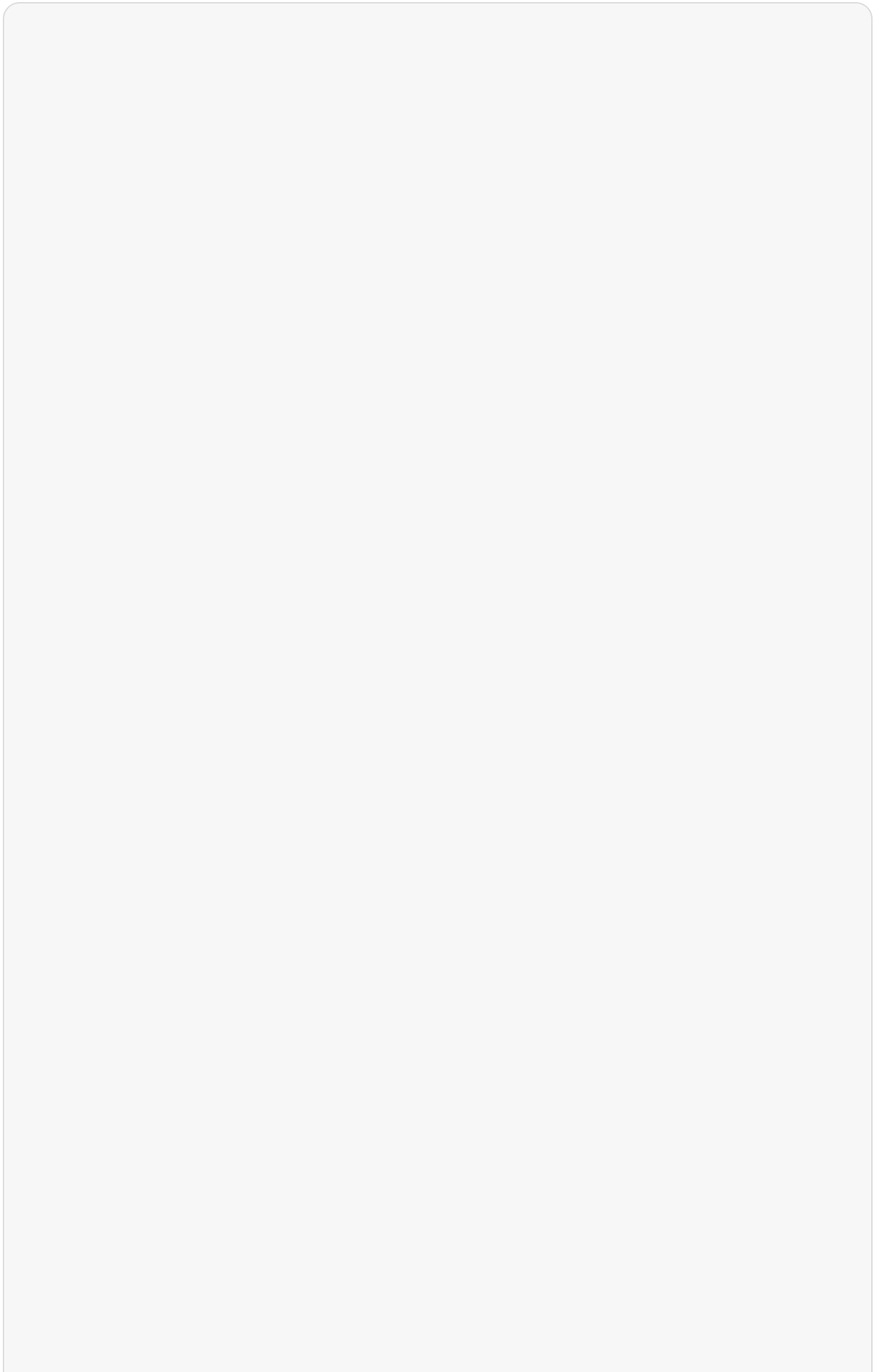
## 37. Alternate layout with Settings



Settings remain available in alternate layout mode.

---

## 38. Mobile workspace



Continuing Previous... · 14 msgs ▾



- **Memory & notes** – persistent memory, notes/checklists, calendar, tasks
- **Model serving** – launch and manage LLMs via vLLM, SGLang, llama.cpp, Ollama
- **API integrations** – connect to external services (RSS, git forge, bookmarks, smart home, etc.)
- **Internal app** – talk to various sub-systems through an app API (cookbook, gallery, documents, settings, research, compare, etc.)

### Stateful features

- **Memory** – persistent across sessions, organized by category (fact, event, contact, preference)
- **Notes** – Google Keep-style notes and checklists with due dates, colors, labels
- **Skills** – reusable procedures for domain-specific workflows (like the ones you saw)
- **Presets** – saved configurations for things like model servers

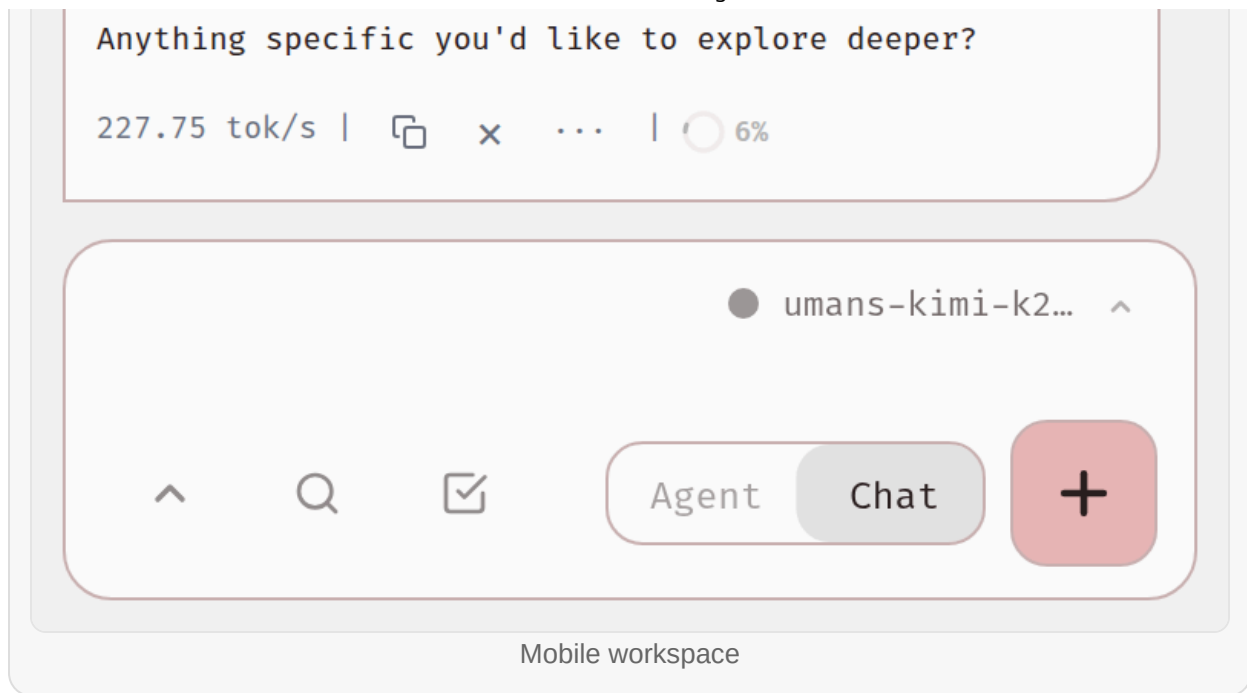
### Infrastructure

- Can connect to remote GPU servers (via SSH) for model serving
- Manages tmux sessions for long-running processes
- Has a model cache for storing downloaded models locally or remotely

### Constraints I operate within

- Step limits per session
- I must use tools to take action (not just describe)
- External source data is treated as untrusted input, not instructions
- I'm designed to be helpful but also safety-conscious about injection attempts

That's pretty much the full picture. I don't have visibility into the underlying LLM weights, training data, or the actual server infrastructure – I'm a layer on top of it.



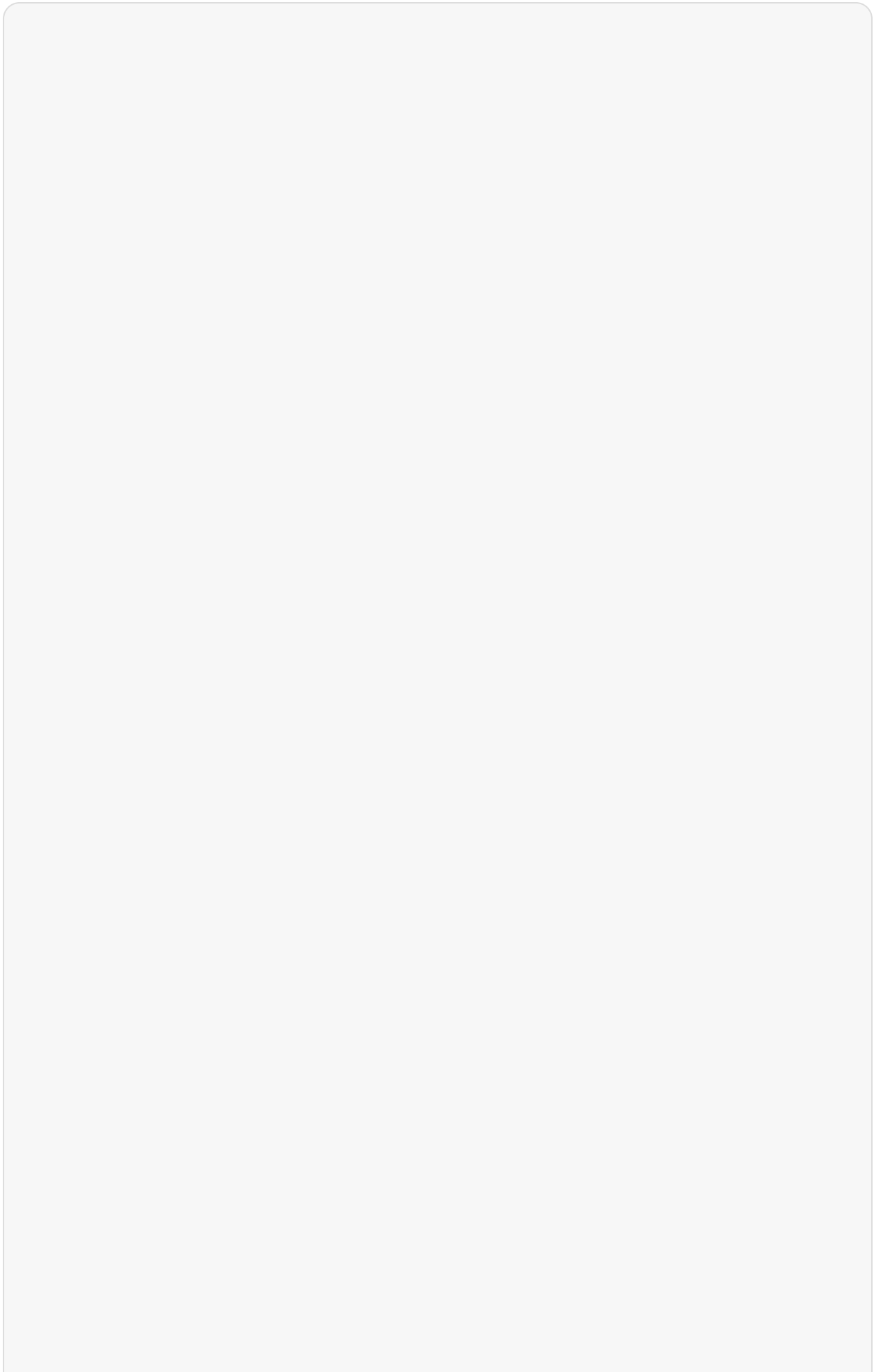
CarbonAgent is responsive and supports mobile-sized screens.

## Mobile notes

- The conversation remains readable.
- The composer adapts to narrow screens.
- Navigation uses mobile controls where available.

---

## 39. Mobile chat and composer



Continuing Previous... · 14 msgs ▾



- **Memory & notes** – persistent memory, notes/checklists, calendar, tasks
- **Model serving** – launch and manage LLMs via vLLM, SGLang, llama.cpp, Ollama
- **API integrations** – connect to external services (RSS, git forge, bookmarks, smart home, etc.)
- **Internal app** – talk to various sub-systems through an app API (cookbook, gallery, documents, settings, research, compare, etc.)

### Stateful features

- **Memory** – persistent across sessions, organized by category (fact, event, contact, preference)
- **Notes** – Google Keep-style notes and checklists with due dates, colors, labels
- **Skills** – reusable procedures for domain-specific workflows (like the ones you saw)
- **Presets** – saved configurations for things like model servers

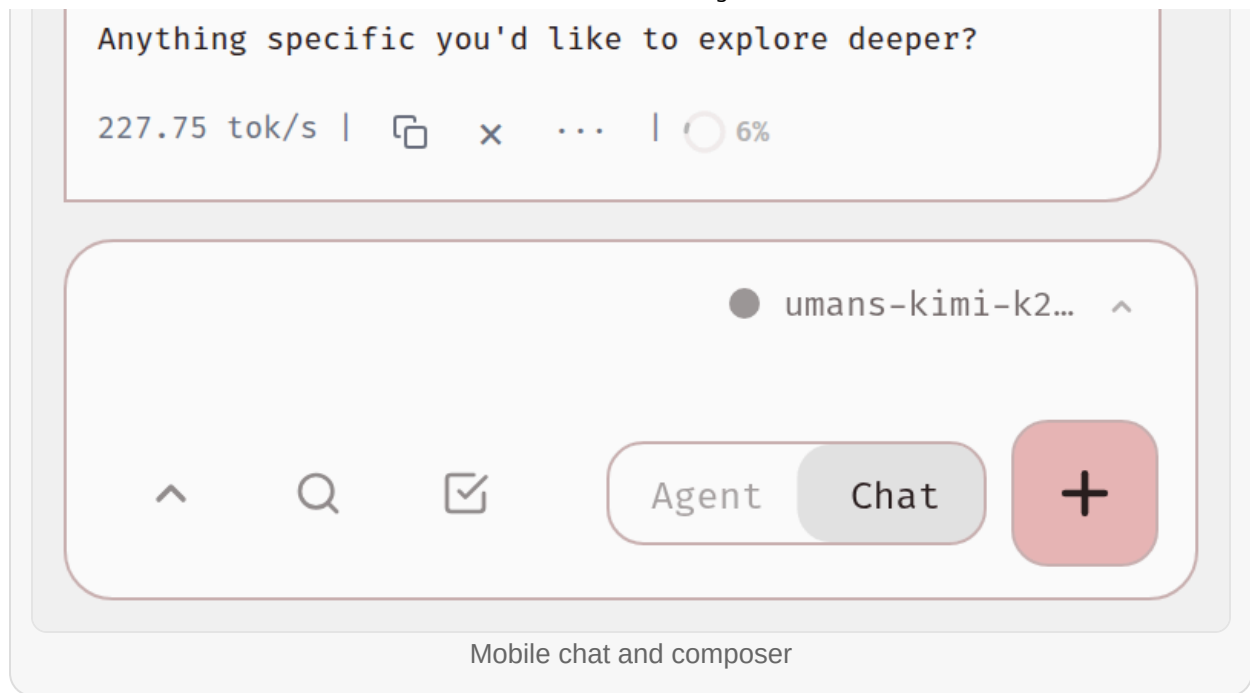
### Infrastructure

- Can connect to remote GPU servers (via SSH) for model serving
- Manages tmux sessions for long-running processes
- Has a model cache for storing downloaded models locally or remotely

### Constraints I operate within

- Step limits per session
- I must use tools to take action (not just describe)
- External source data is treated as untrusted input, not instructions
- I'm designed to be helpful but also safety-conscious about injection attempts

That's pretty much the full picture. I don't have visibility into the underlying LLM weights, training data, or the actual server infrastructure – I'm a layer on top of it.



This screenshot shows the mobile chat area and composer controls at phone width.

---

## Practical workflows

### Configure a model

1. Open **Settings**.
2. Go to **Add Models**.
3. Add a local or API endpoint.
4. Confirm the model appears in the model picker.
5. Select it before chatting.

### Run an agent task against local files

1. Attach a workspace folder if needed.
2. Enable **Agent** mode.
3. Enable **Shell access** only if commands are required.
4. Use **Plan mode** for safer multi-step work.
5. Send a specific instruction.

### Compare models

1. Open **Compare**.
2. Select models.
3. Enter a test prompt.
4. Run the comparison.
5. Pick the best model for the task.

### Save reusable context

1. Open **Brain**.
2. Add durable facts as memories.
3. Add repeatable procedures as skills.
4. Tidy/delete stale entries periodically.

### Download and serve local models

1. Open **Cookbook**.
2. Use hardware scan/download recommendations.

3. Download a compatible model.
  4. Serve the model.
  5. Select it from the model picker.
-

---

## Safety and maintenance notes

- Use **Chat** mode for conversation.
  - Use **Agent** mode for tool-using workflows.
  - Enable **Shell access** only when local command execution is required.
  - Use **Plan mode** before risky changes.
  - Keep memories and skills clean and specific.
  - Disable unused integrations or model endpoints.
  - Prefer focused tasks over giant “do everything” prompts.
-

## Screenshot inventory

#	Screenshot	Feature
1	01-chat-workspace.png	Default chat workspace
2	02-model-picker.png	Model picker
3	03-agent-tools.png	Agent/chat tools
4	04-overflow-tools.png	Overflow tools
5	05-prompt-inject.png	Prompt injection
6	06-persona.png	Persona controls
7	07-group-chat.png	Group chat
8	08-conversation-search.png	Conversation search
9	09-brain-memory.png	Brain memories
10	10-brain-skills.png	Brain skills
11	11-brain-add.png	Add memory/skill
12	12-calendar.png	Calendar
13	13-compare.png	Compare
14	14-cookbook-download.png	Cookbook download
15	15-cookbook-serve.png	Cookbook serve
16	16-cookbook-dependencies.png	Cookbook dependencies
17	17-cookbook-settings.png	Cookbook settings
18	18-deep-research.png	Deep Research
19	19-gallery.png	Gallery
20	20-library-documents.png	Library/Documents
21	21-notes.png	Notes
22	22-tasks.png	Tasks

#	Screenshot	Feature
23	23-theme-browse.png	Theme browser
24	24-theme-customize.png	Theme customization
25	25-settings-add-models.png	Settings: Add Models
26	26-settings-ai-defaults.png	Settings: AI Defaults
27	27-settings-search.png	Settings: Search
28	28-settings-integrations.png	Settings: Integrations
29	29-settings-email.png	Settings: Email
30	30-settings-appearance.png	Settings: Appearance
31	31-settings-shortcuts.png	Settings: Shortcuts
32	32-settings-account.png	Settings: Account
33	33-email.png	Email
34	34-layout-mode-workspace.png	Alternate layout workspace
35	35-layout-mode-command-deck.png	Alternate layout command deck
36	36-layout-mode-cookbook.png	Alternate layout Cookbook
37	37-layout-mode-settings.png	Alternate layout Settings
38	38-mobile-workspace.png	Mobile workspace
39	39-mobile-navigation.png	Mobile chat/composer